Advancde Deep Learning

1. Variational autoencoders *

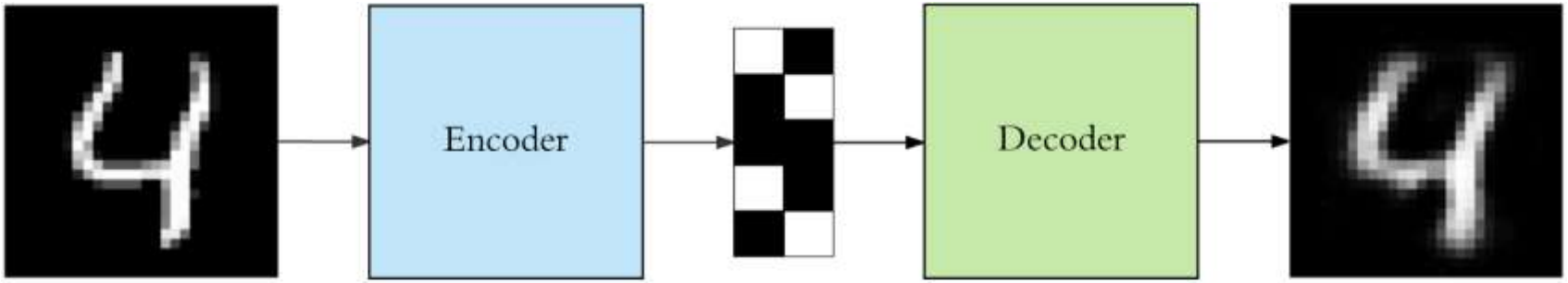
Prof. Jianguo Zhang

SUSTech

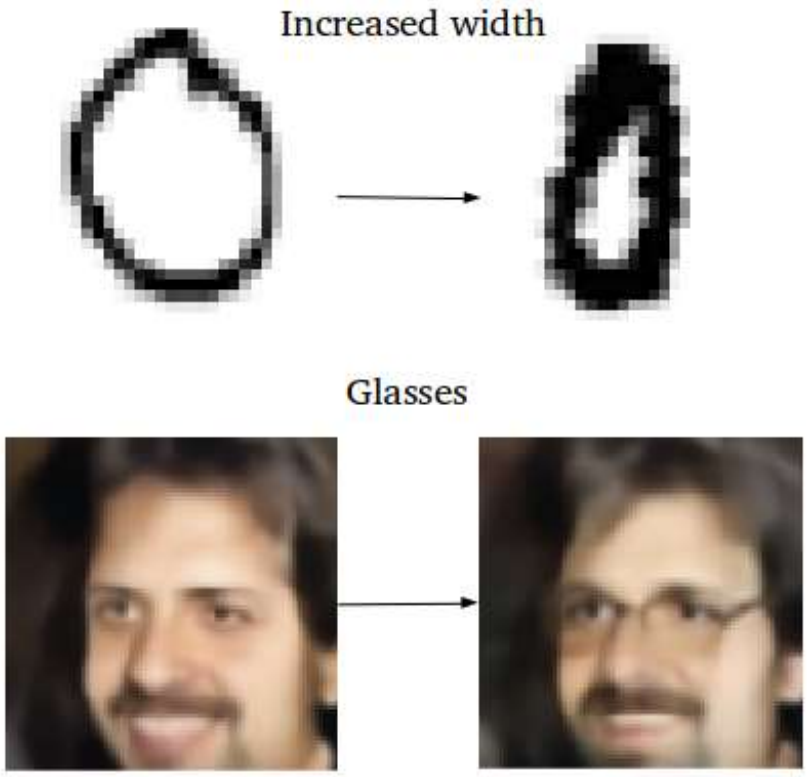
*Based on https://jaan.io/what-is-variational-autoencoder-vae-tutorial/ and http://www.deeplearningbook.org chapter 20.10.3

Generation with autoencoders?

- Can we use an autoencoder to generate data given a code?



Ok. But why? Sample application



Exploring a specific variation of input data

Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

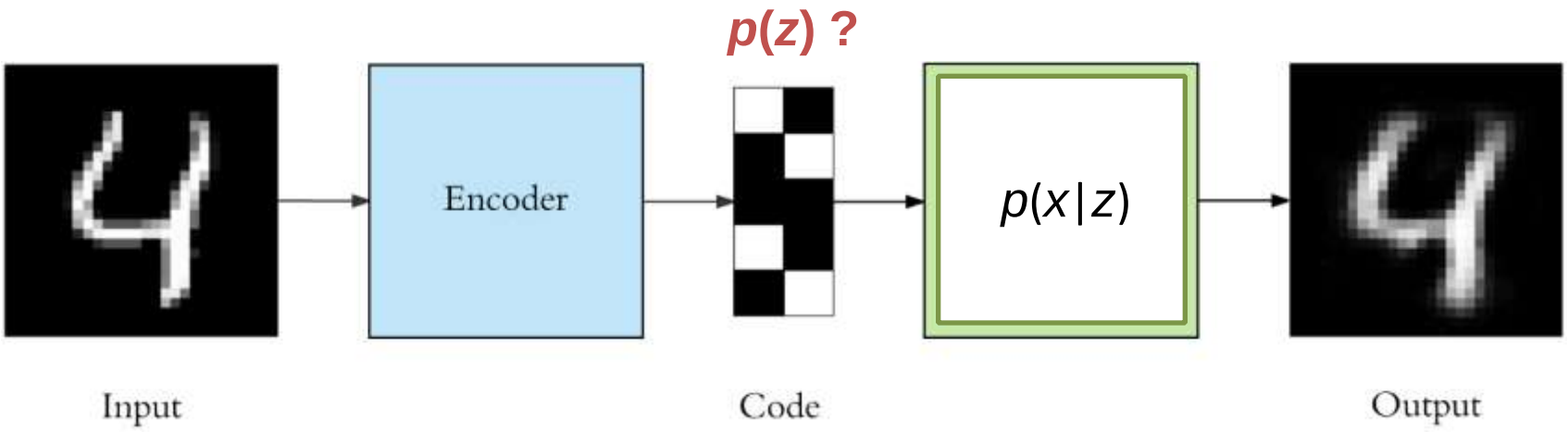
Ok. But why? Sample application

Exploring a specific variation of input data

Source: Sampling Generative Networks https://arxiv.org/pdf/1609.04468.pdf

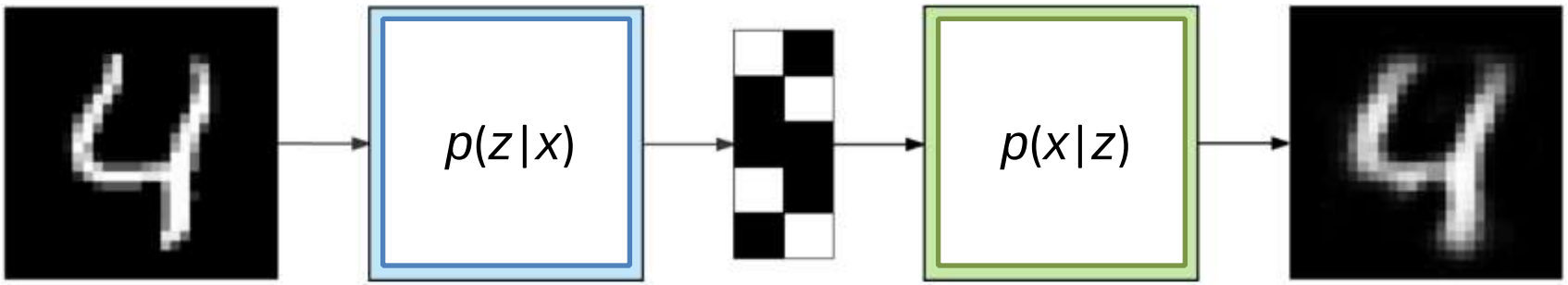
Generation with autoencoders?

- Can we use an autoencoder to generate data given a code?
- Yes, but…how to sample the code z first?



Generation with autoencoders?

- Can we use an autoencoder to generate data given a code?
- Yes, but…how to sample the code z first?

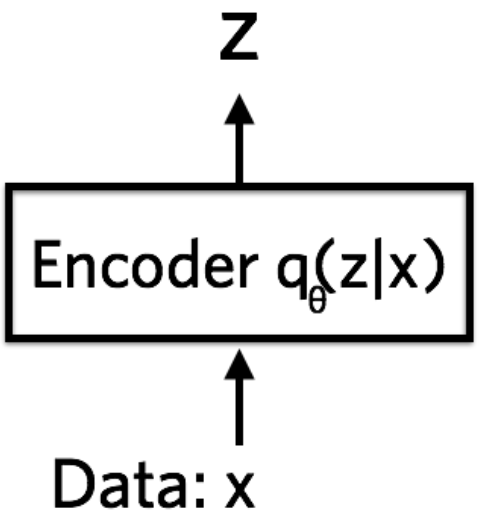


Variational autoencoder

- Let’s have a closer look at our (variational) autoencoder components

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



The Encoder is a neural network that takes a data point in input and outputs a hidden representation **z**, usually lower dimensional

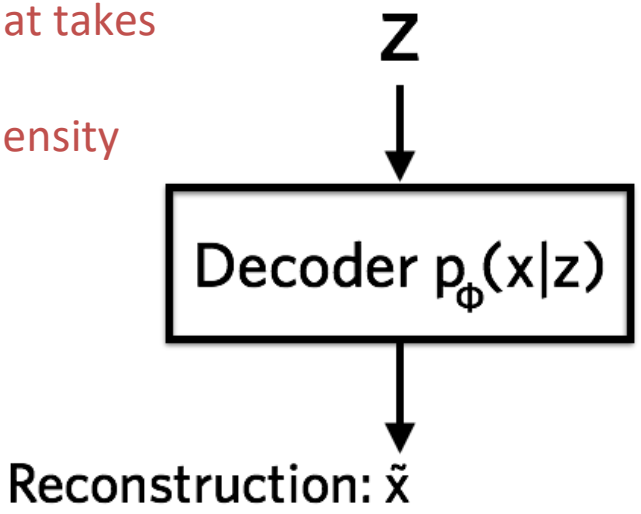
More precisely, the Encoder outputs the parameters of a Gaussian probability density $\mathbf{q_\theta(z|x)}$

Then, we can sample a representation/code **z** from $\mathbf{q_\theta(z|x)}$

VAEs: neural networks perspective

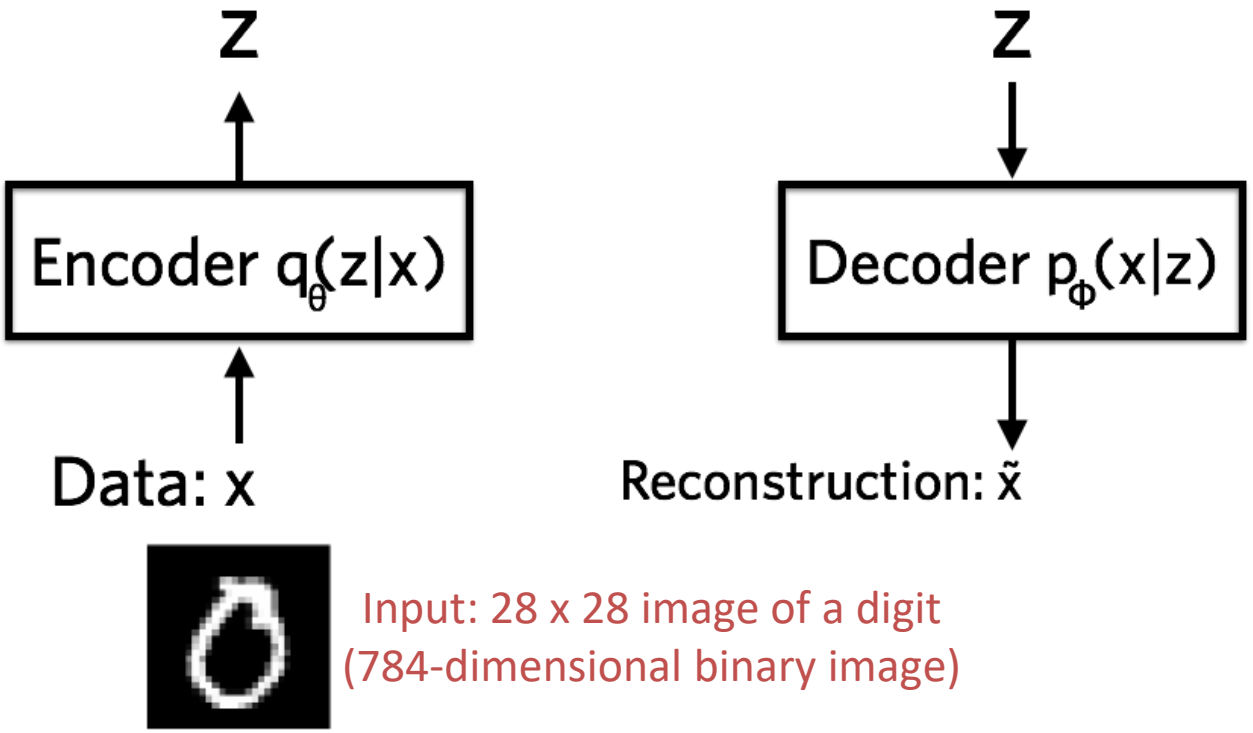
- Let's have a closer look at our (variational) autoencoder components

The Decoder is another neural network that takes the representation **z** in input and outputs the parameters of $\mathbf{p_\phi(x|z)}$, a probability density from which we can sample the data



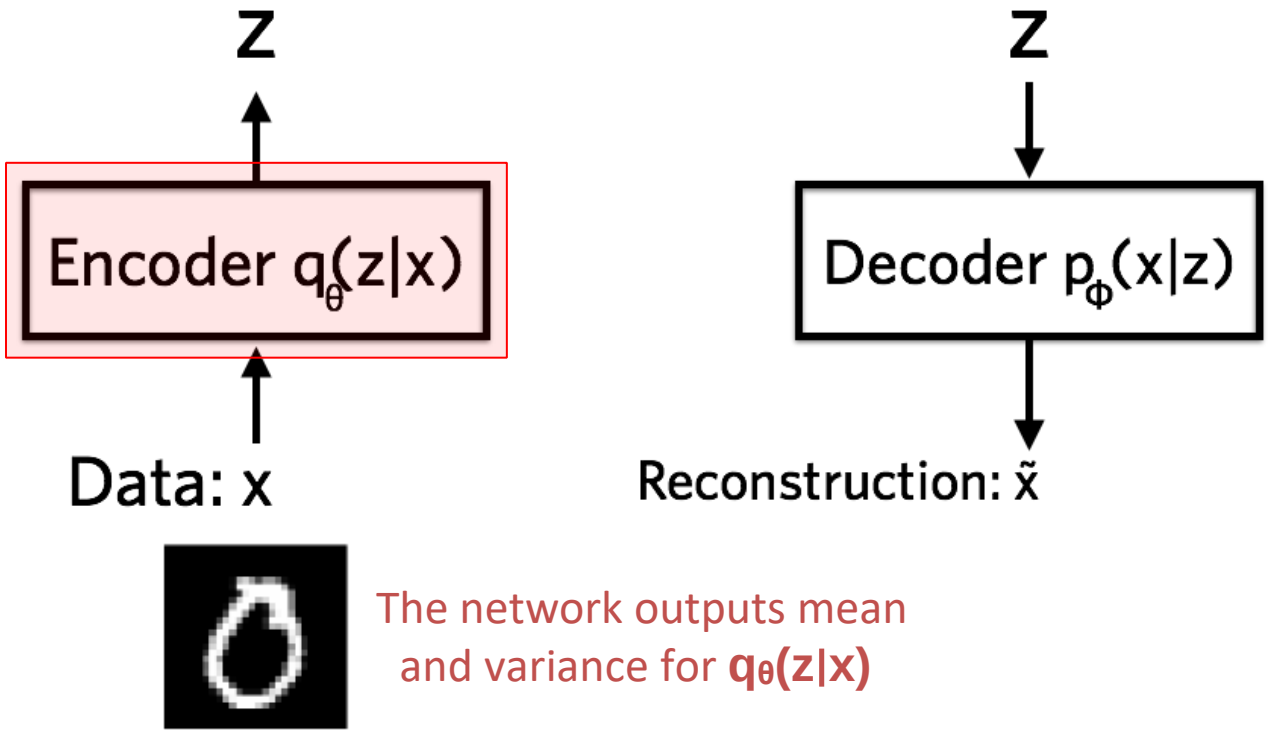
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



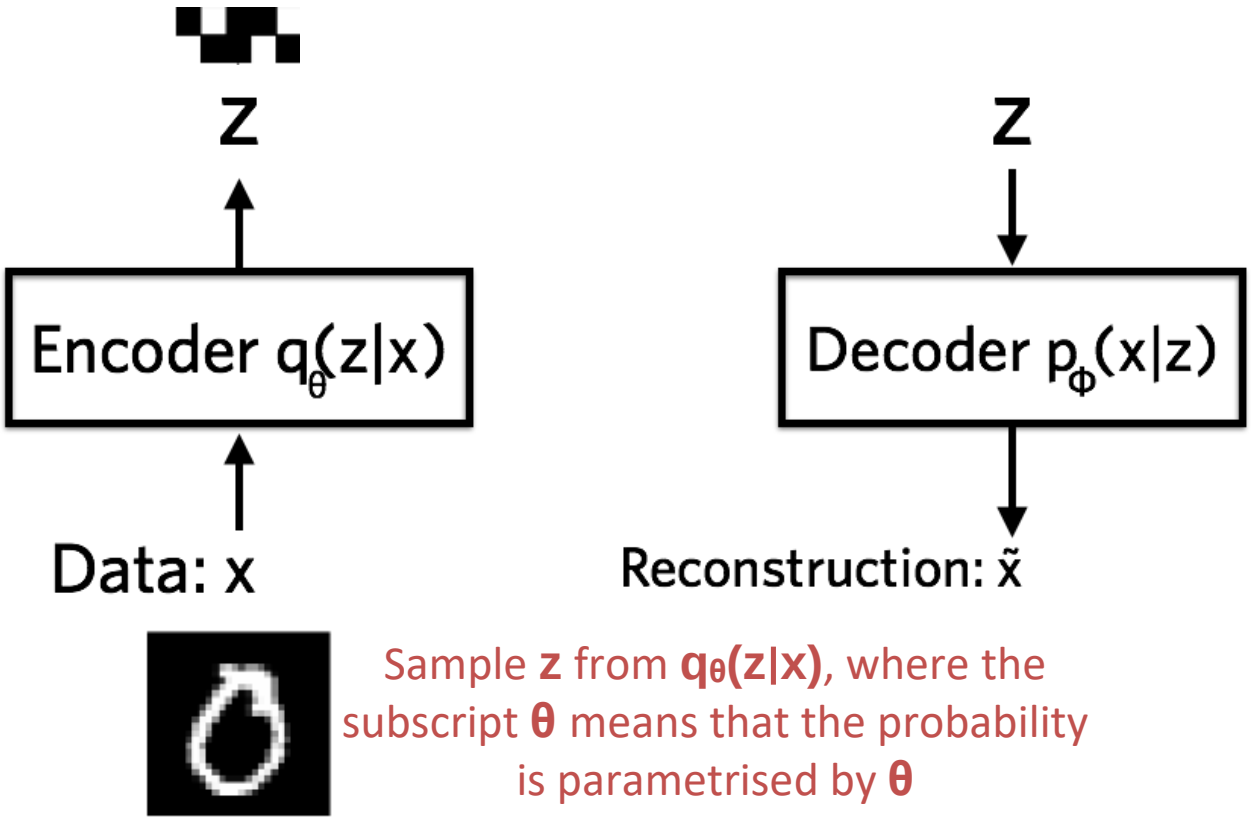
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



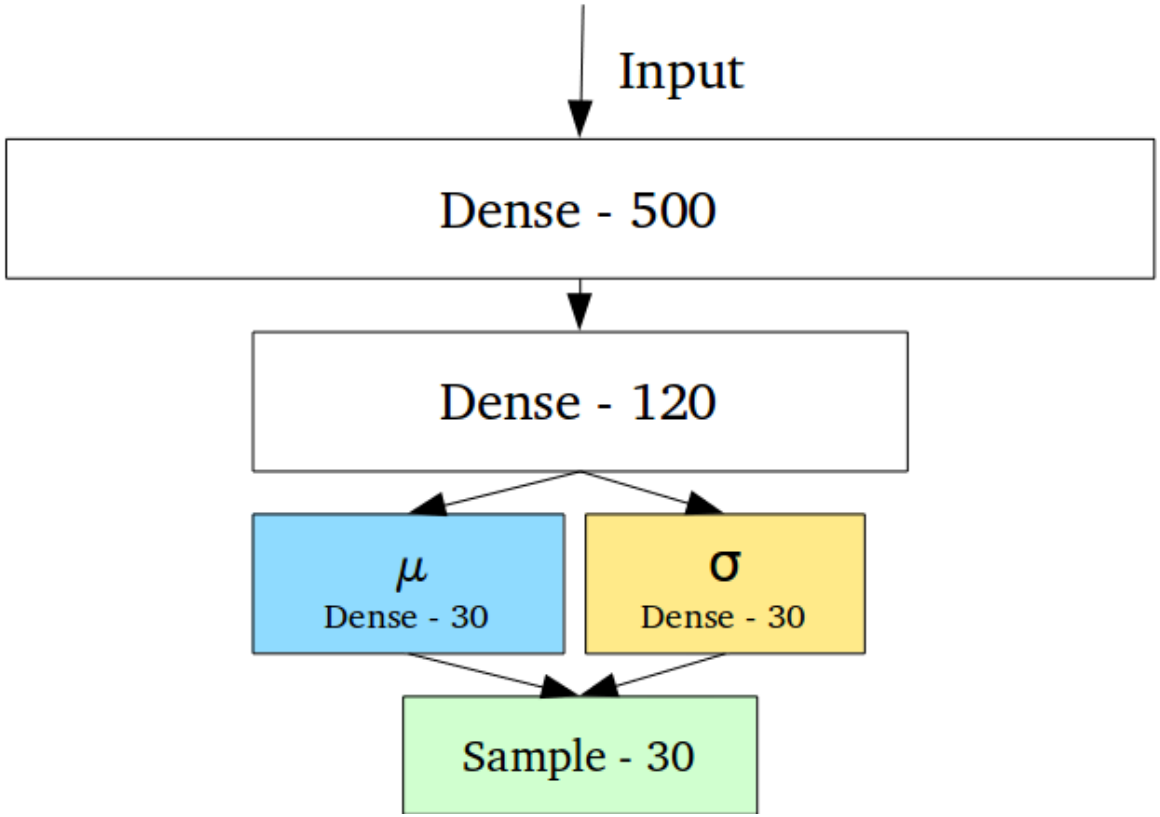
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



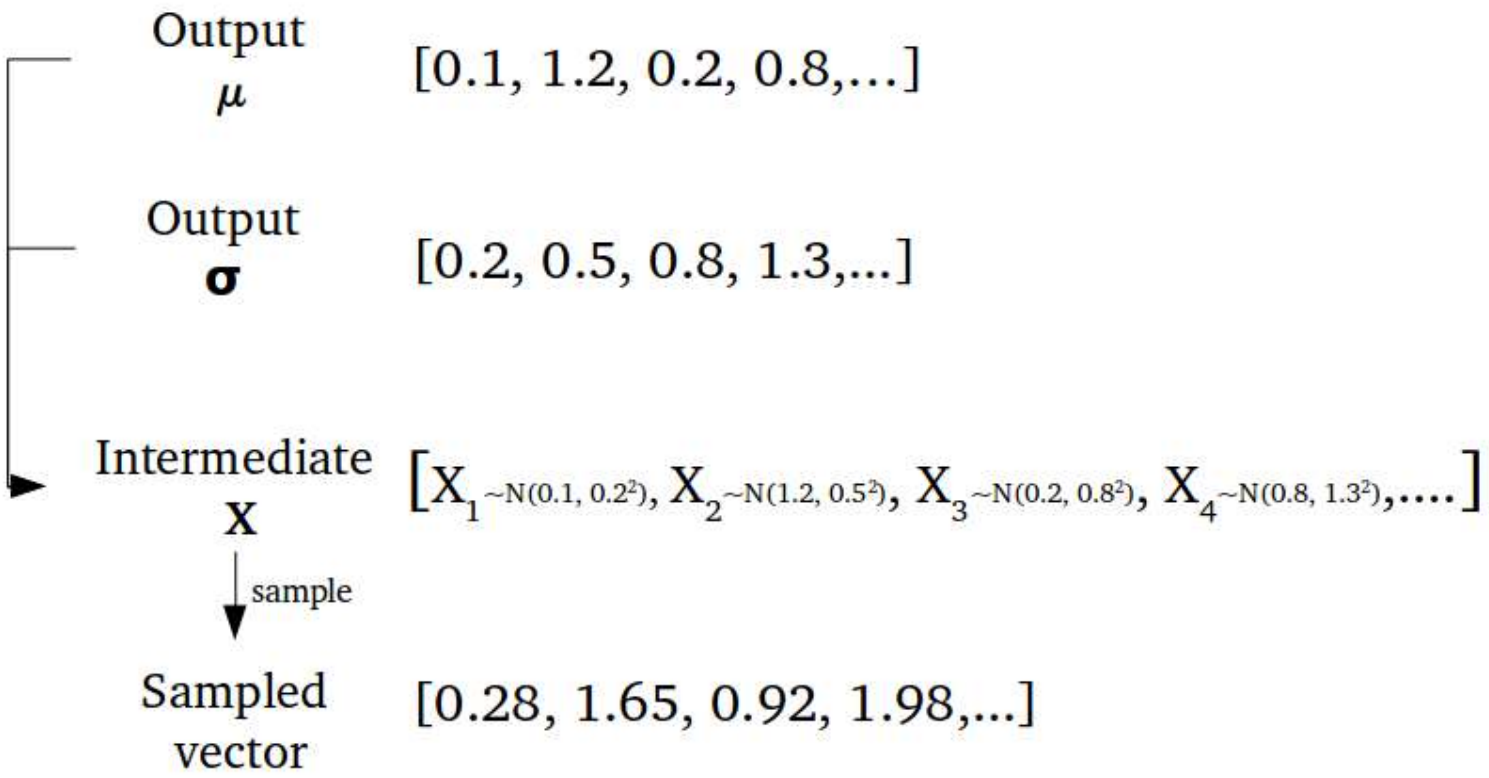
Sample **z** from $q_\theta(z|x)$, where the subscript **θ** means that the probability is parametrised by **θ**

VAEs: encoder



Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

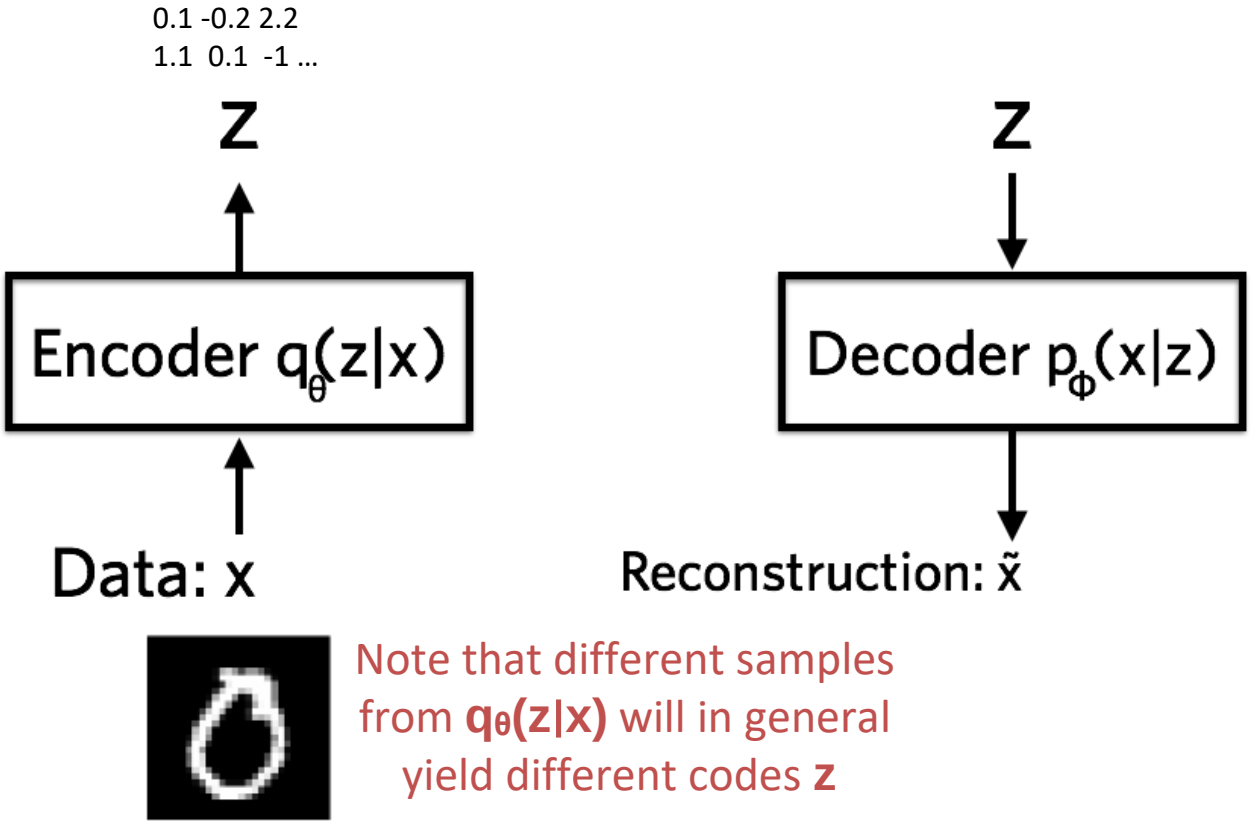
VAEs: encoder



Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

VAEs: neural networks perspective

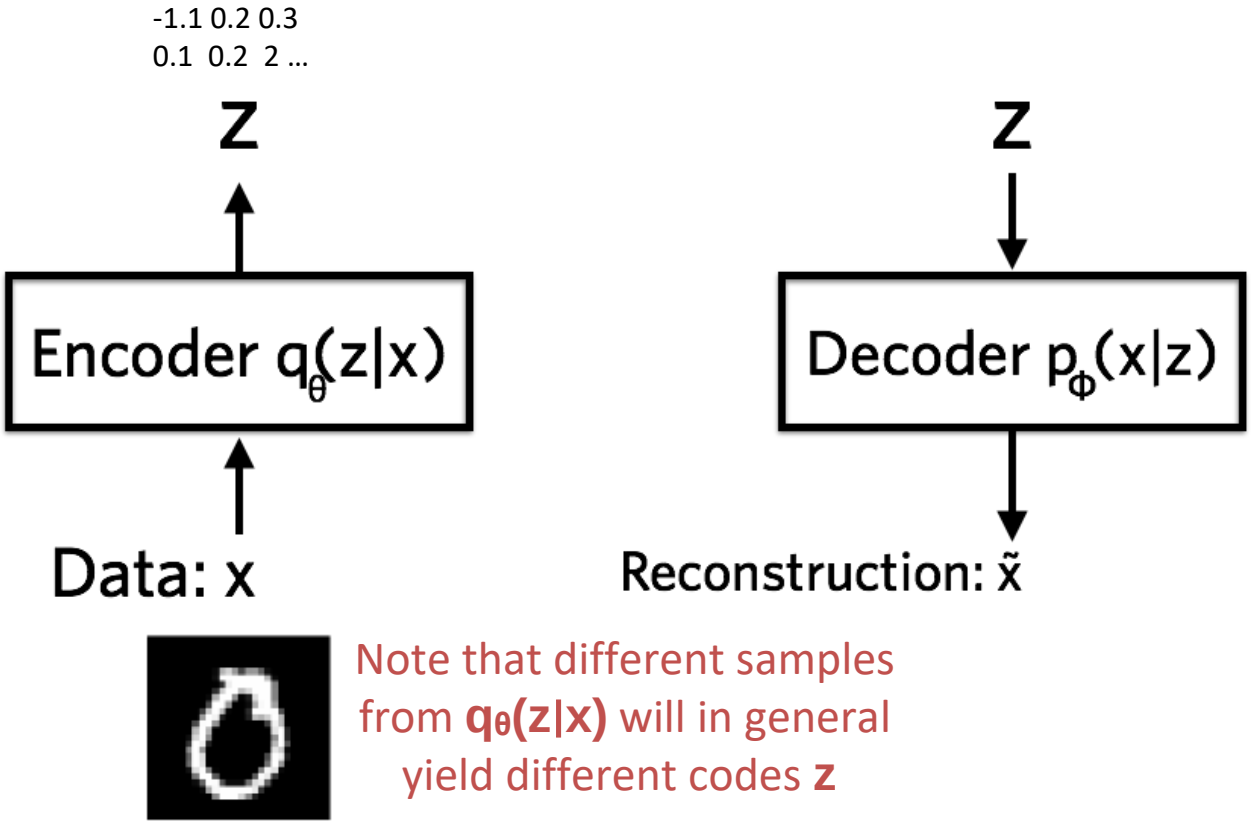
- Let's have a closer look at our (variational) autoencoder components



Note that different samples from $\mathbf{q_\theta(z|x)}$ will in general yield different codes $\mathbf{z}$

VAEs: neural networks perspective

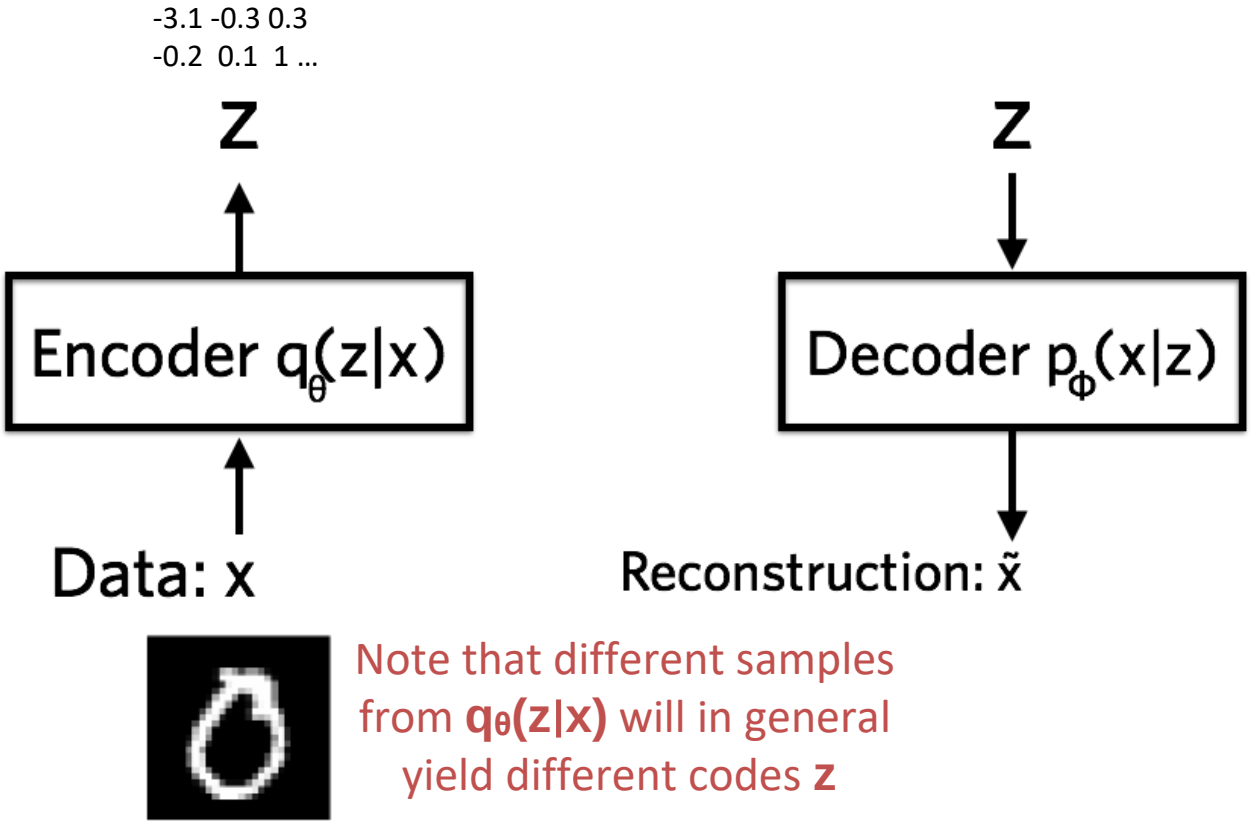
- Let's have a closer look at our (variational) autoencoder components



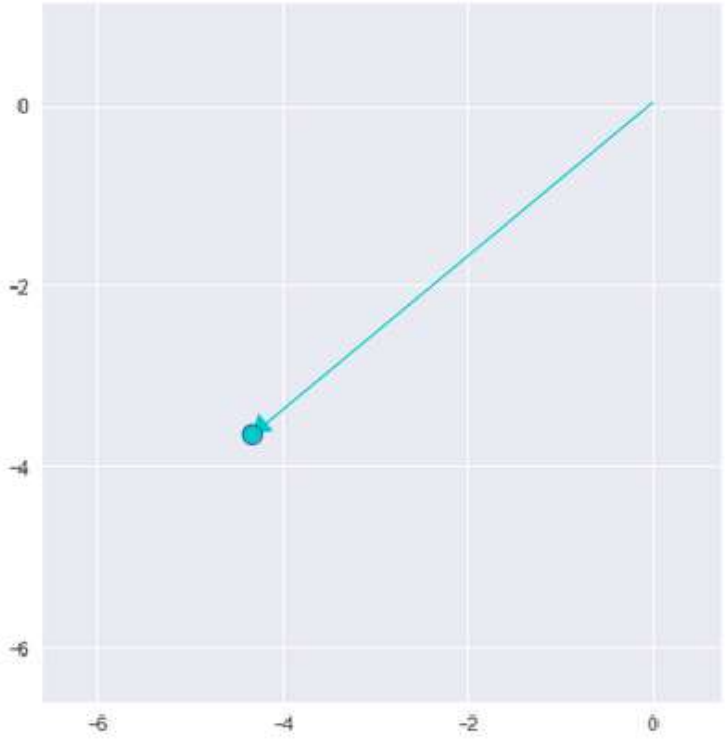
Note that different samples from $\mathbf{q_\theta(z|x)}$ will in general yield different codes **z**

VAEs: neural networks perspective

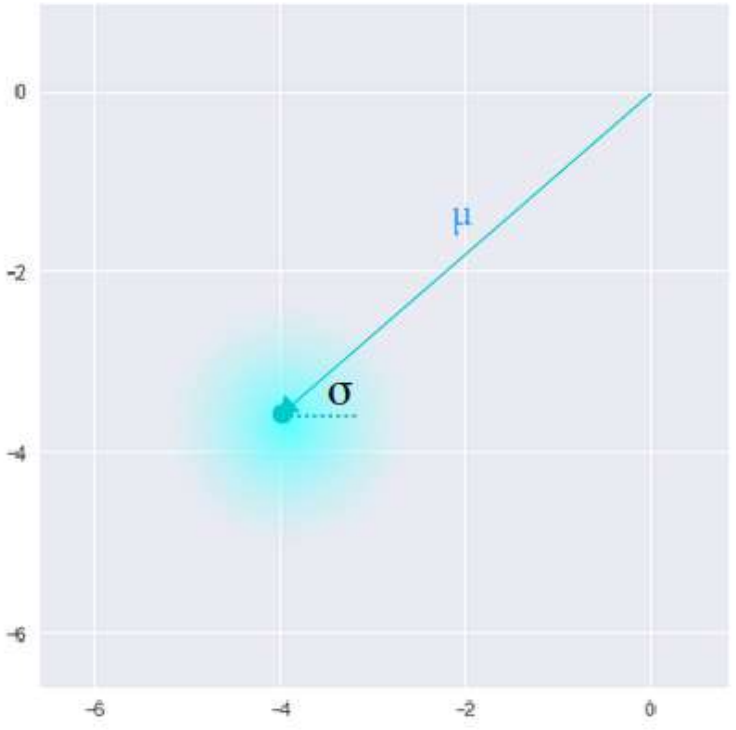
- Let's have a closer look at our (variational) autoencoder components



Latent space: AE vs VAEs



Standard Autoencoder
(direct encoding coordinates)

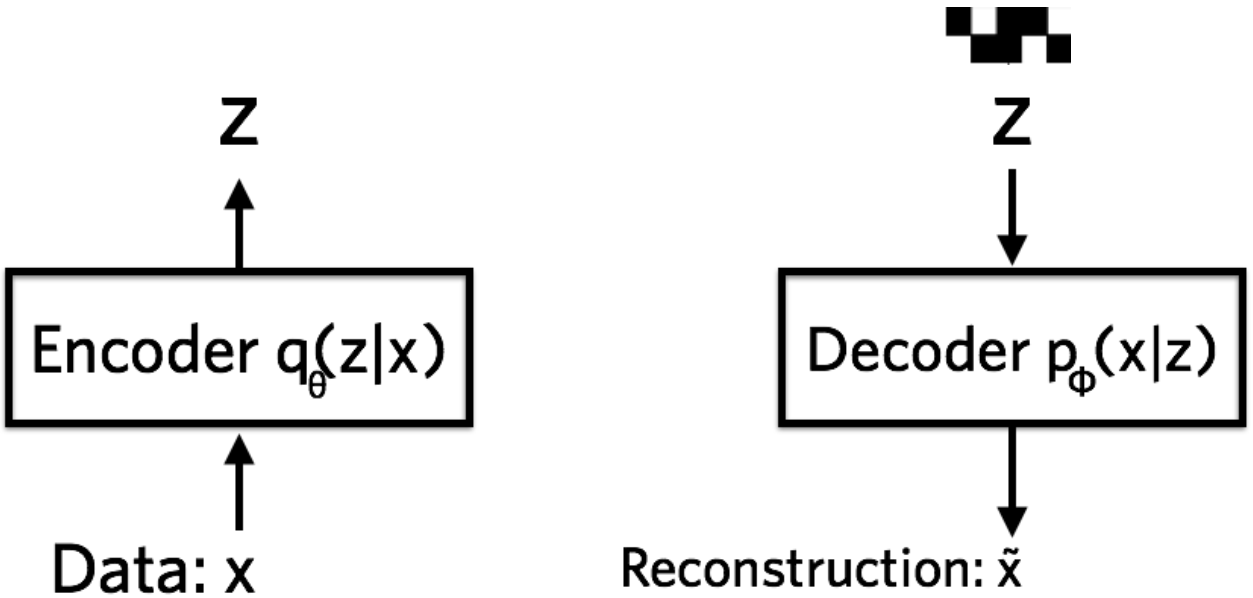


Variational Autoencoder
(μ and σ initialize a probability distribution)

Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

VAEs: neural networks perspective

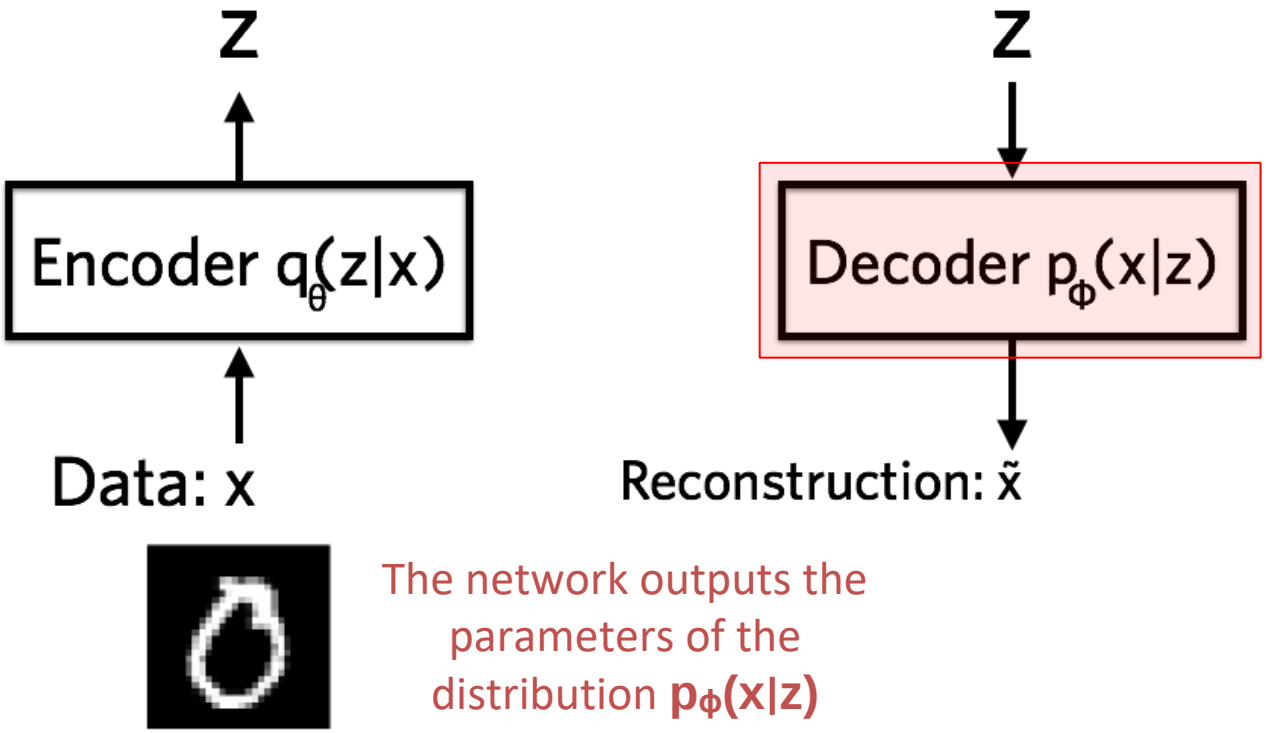
- Let's have a closer look at our (variational) autoencoder components



z is given in input to another neural network (the Decoder) with its weights and biases (**ϕ**)

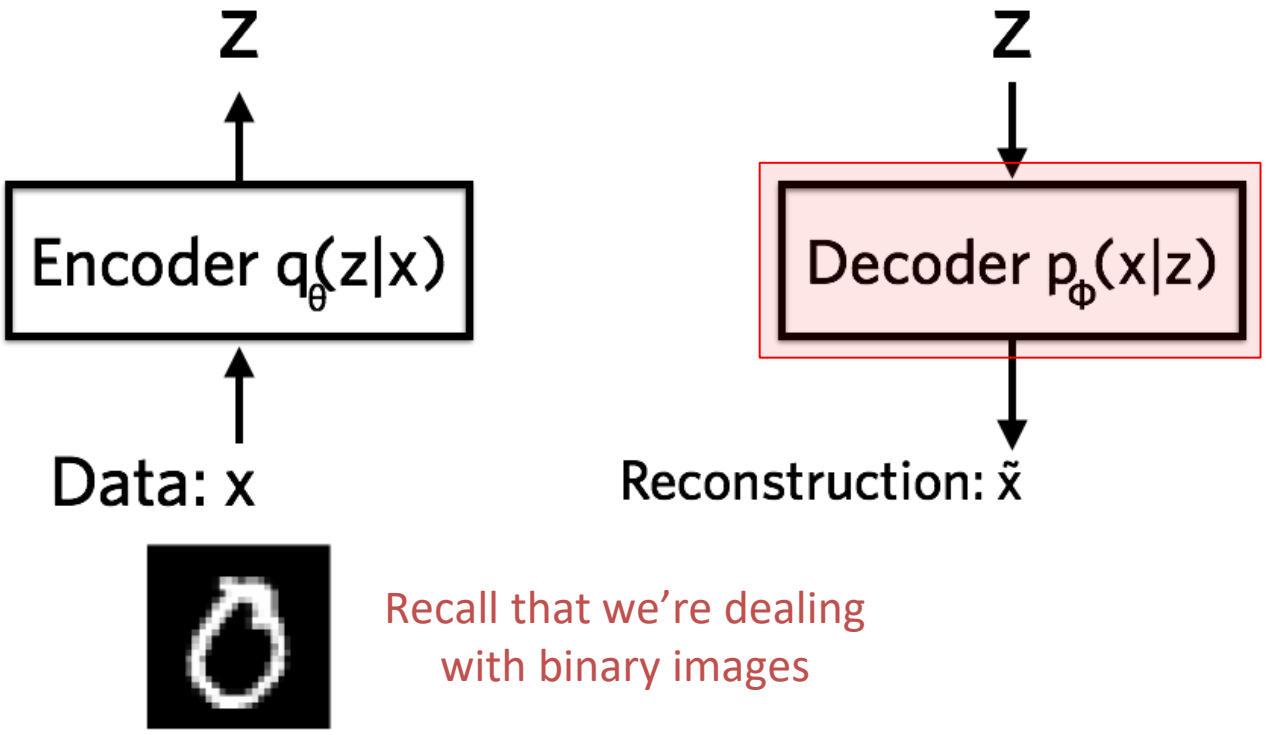
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



VAEs: neural networks perspective

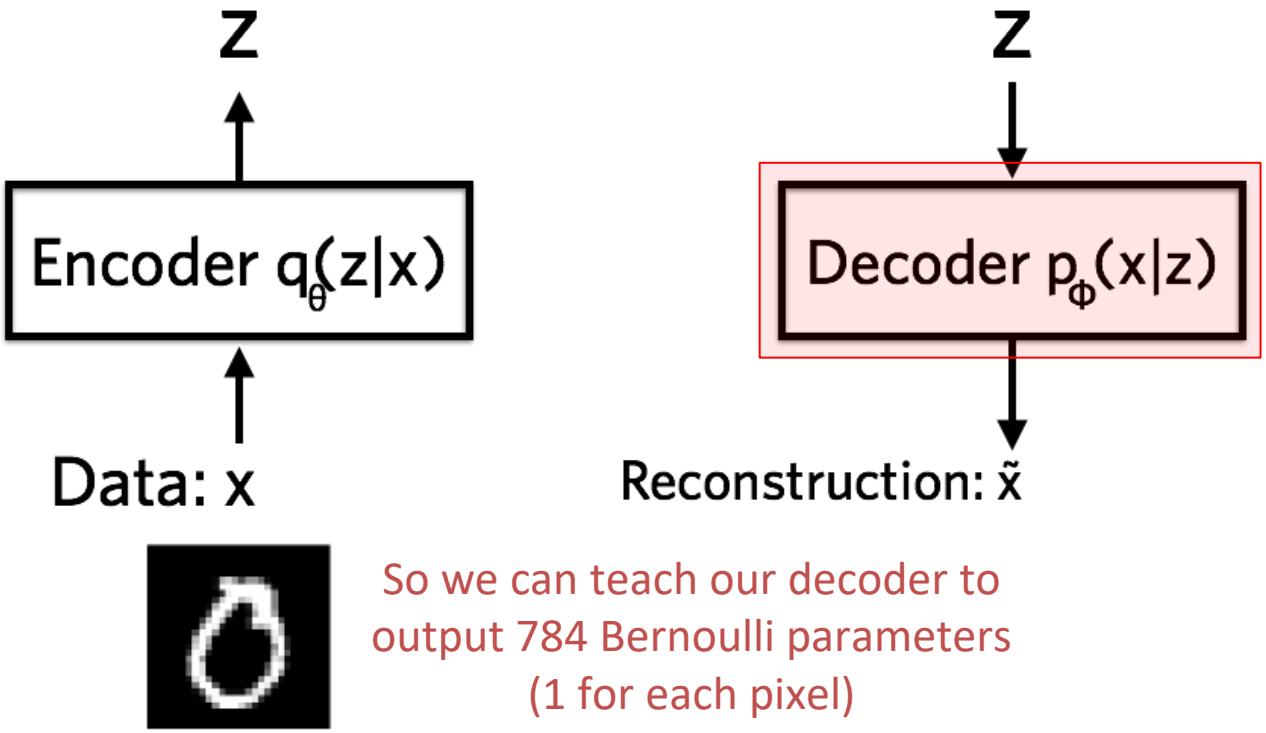
- Let's have a closer look at our (variational) autoencoder components



Recall that we're dealing with binary images

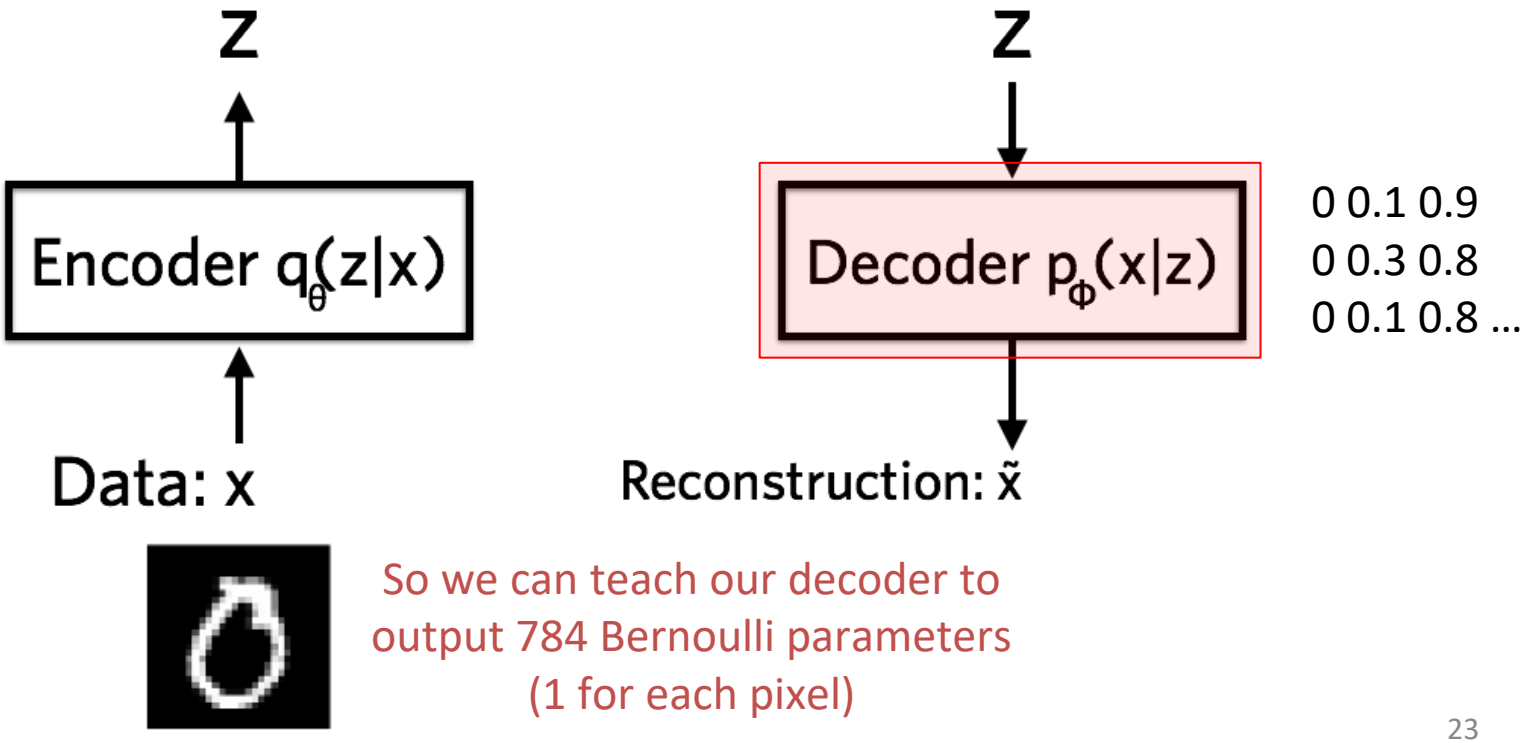
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



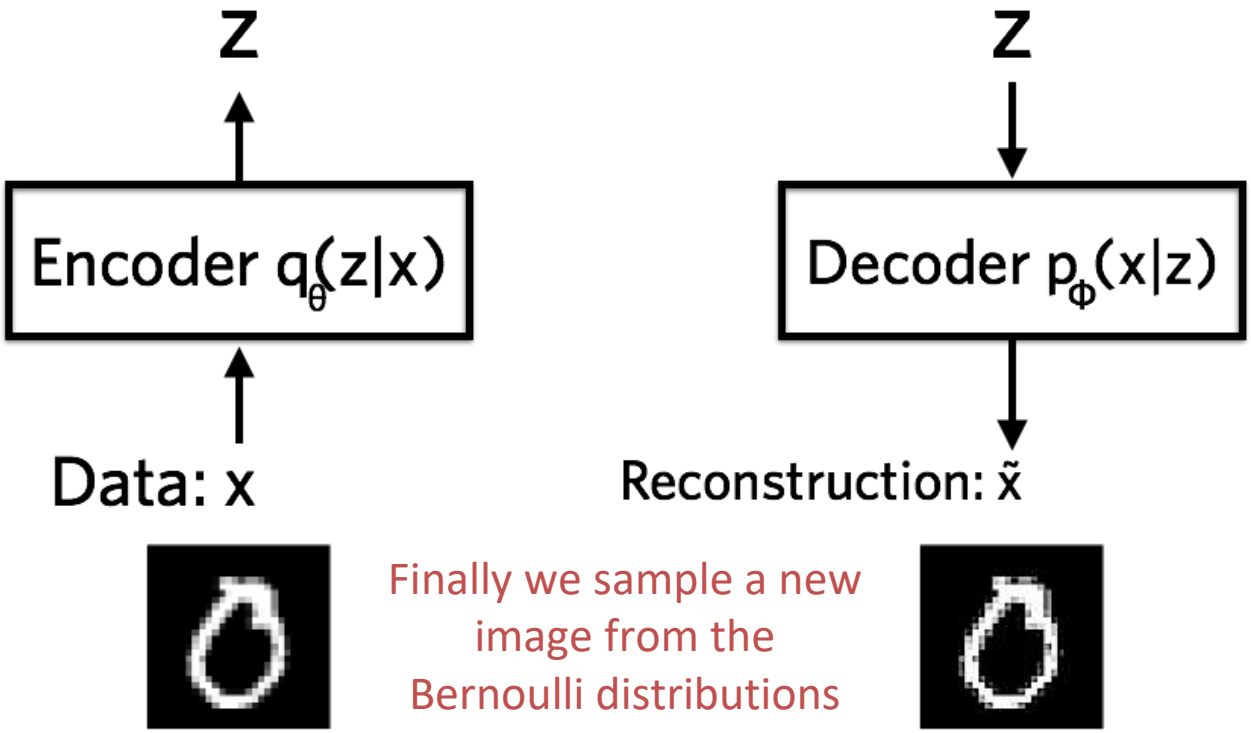
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



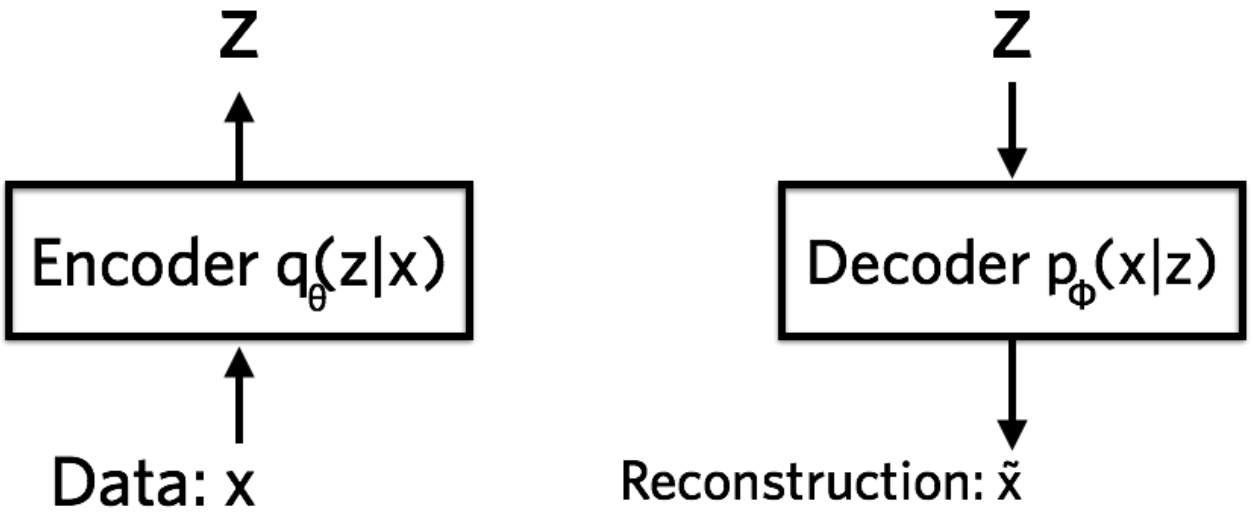
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



VAEs: neural networks perspective

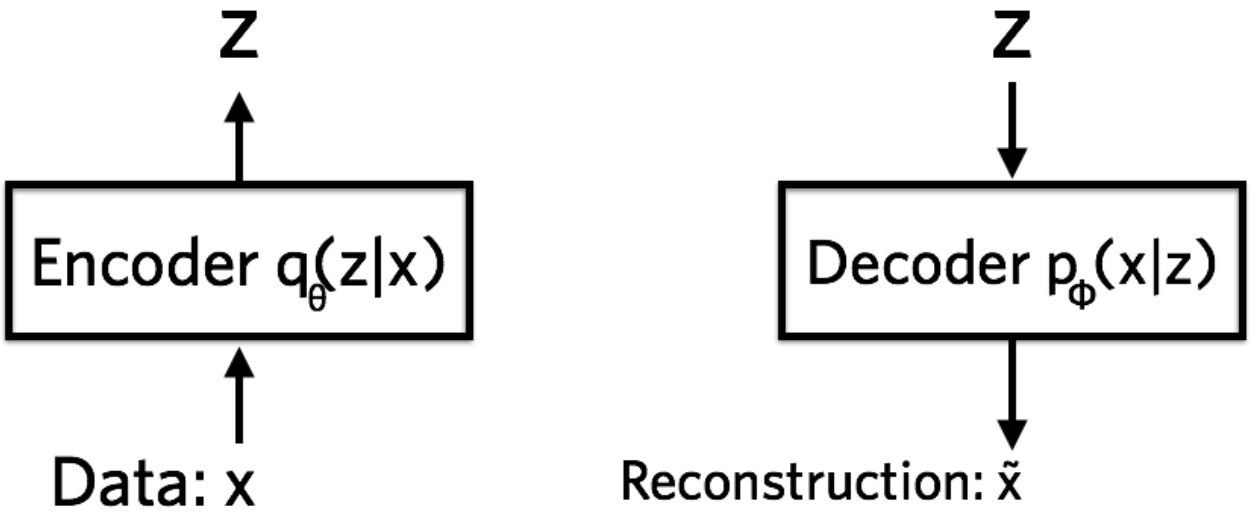
- Let's have a closer look at our (variational) autoencoder components



How much information is lost when going from the low-dimensional representation **z** to the higher dimensional reconstructed **x**?

VAEs: neural networks perspective

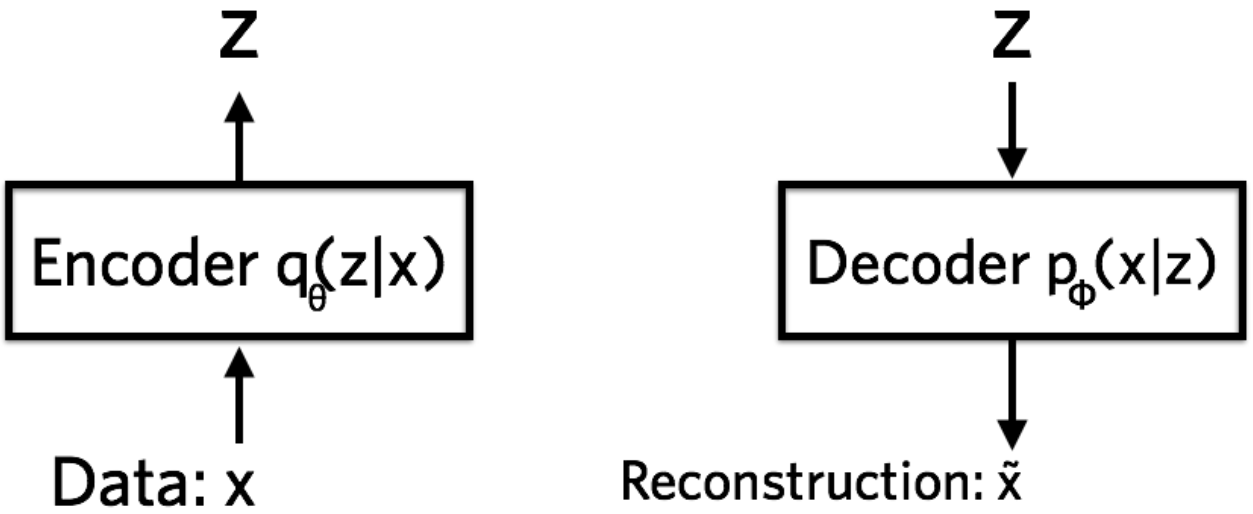
- Let's have a closer look at our (variational) autoencoder components



This can be measured using the reconstruction log-likelihood **$\log p_\phi(x|z)$**
It tells us how effectively the decoder has learned to reconstruct **x** given **z**

VAEs: neural networks perspective

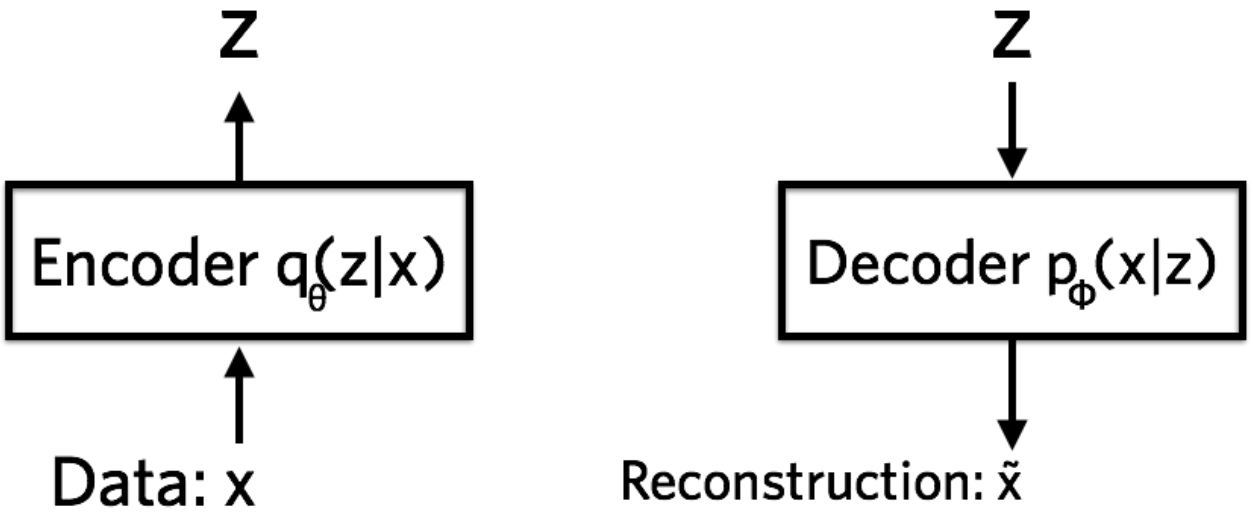
- Let's have a closer look at our (variational) autoencoder components



Sounds nice, but we need a loss if we want to do back-propagation and learn the optimal parameters of the two networks (encoder & decoder)

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

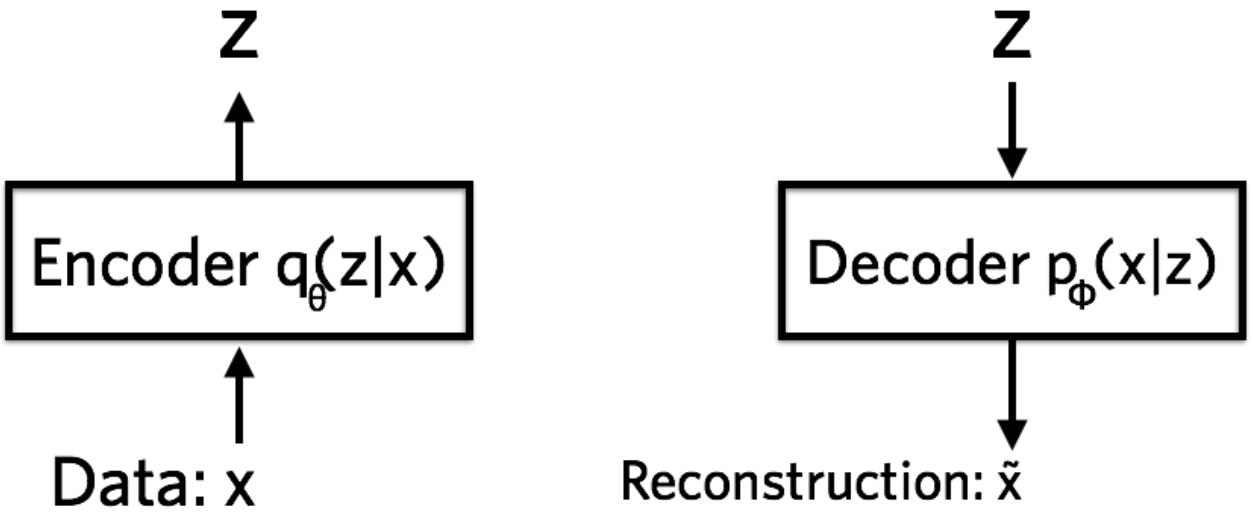


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

loss function for i-th datapoint - the total loss is the sum of all the l_i

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

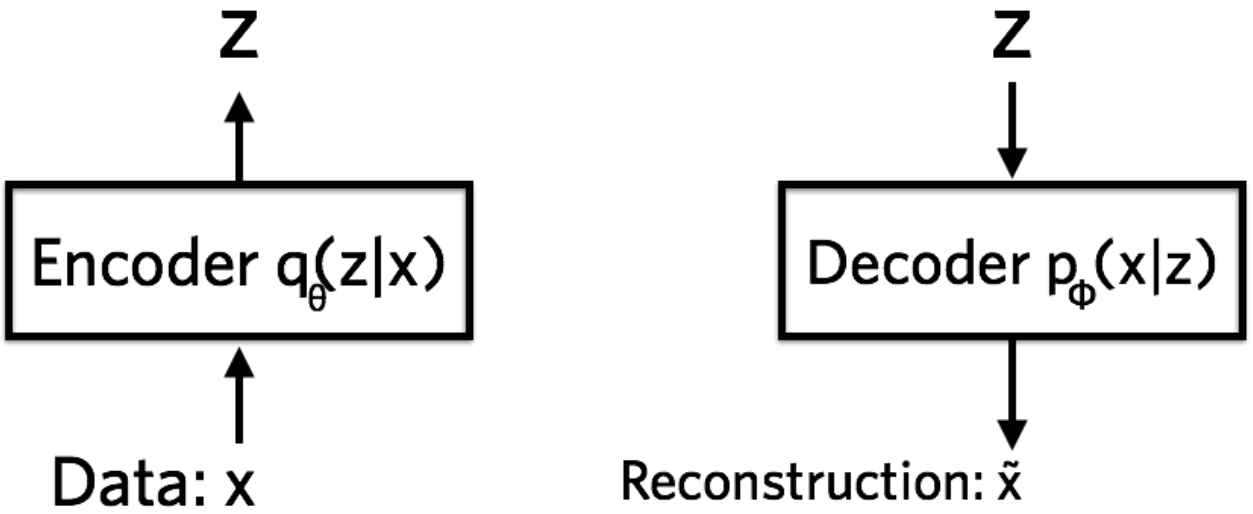


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

Remember, for one data point we can sample many codes **z**

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

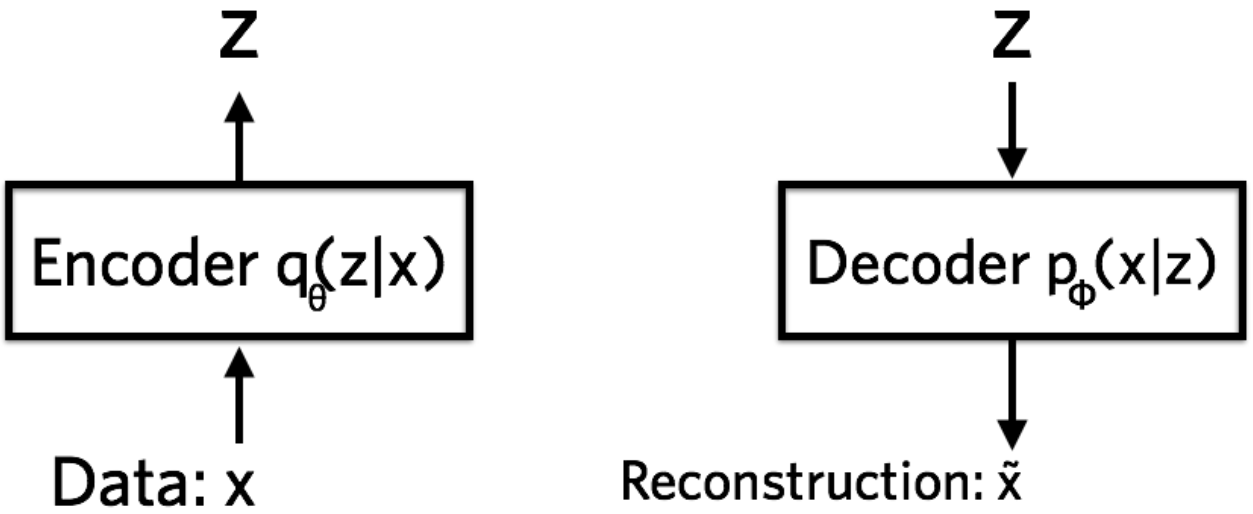


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

This is why here we have the expected log-likelihood (negative, we're minimising)

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

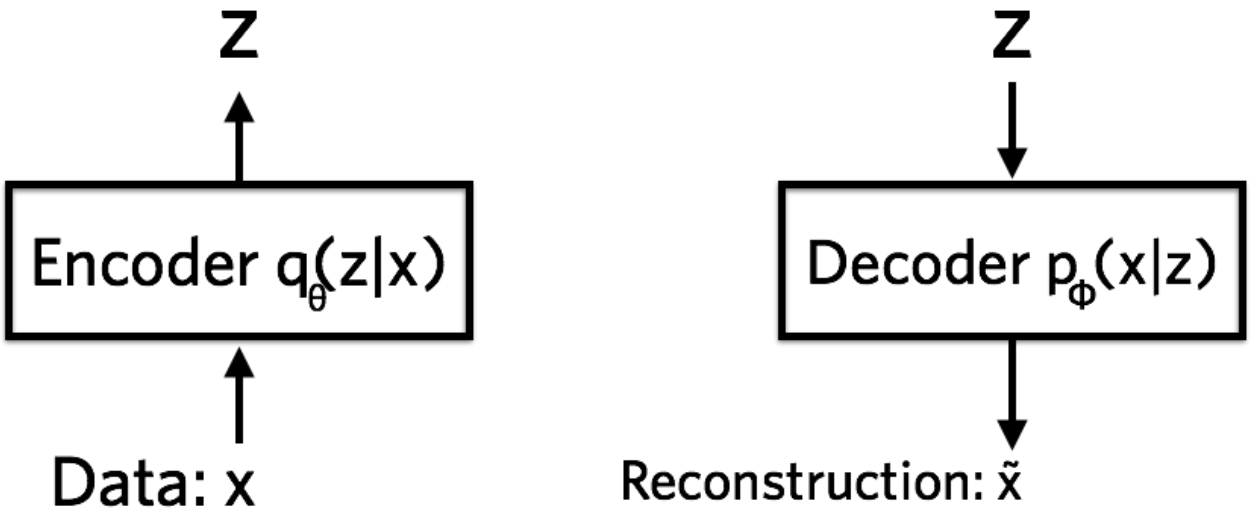


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

This term encourages decoder to learn to reconstruct the data

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

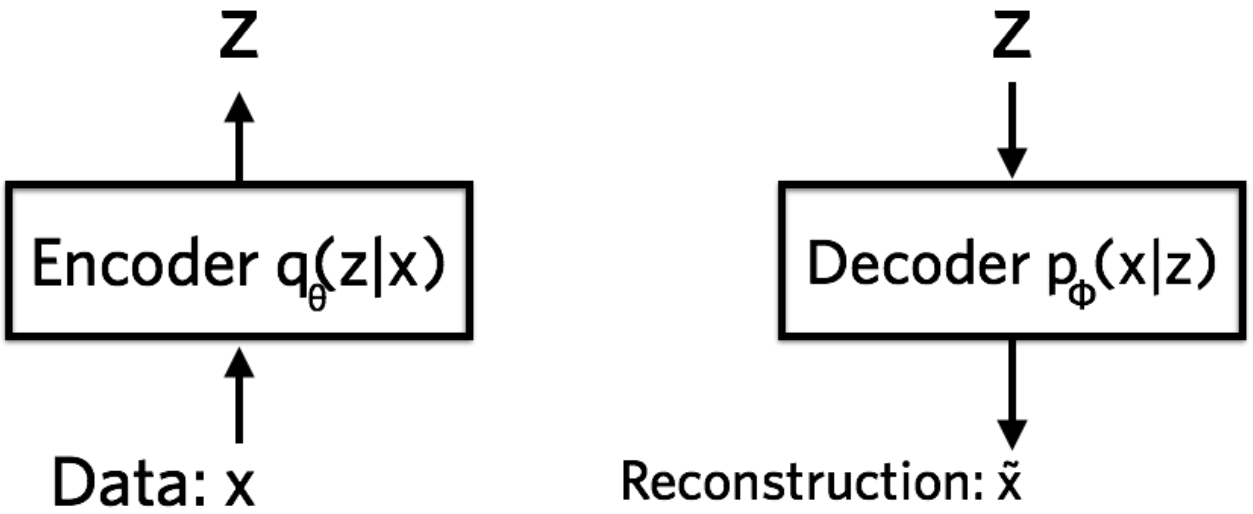


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

Bad encoder (e.g., high prob of black pixel when it should be white) = large cost

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

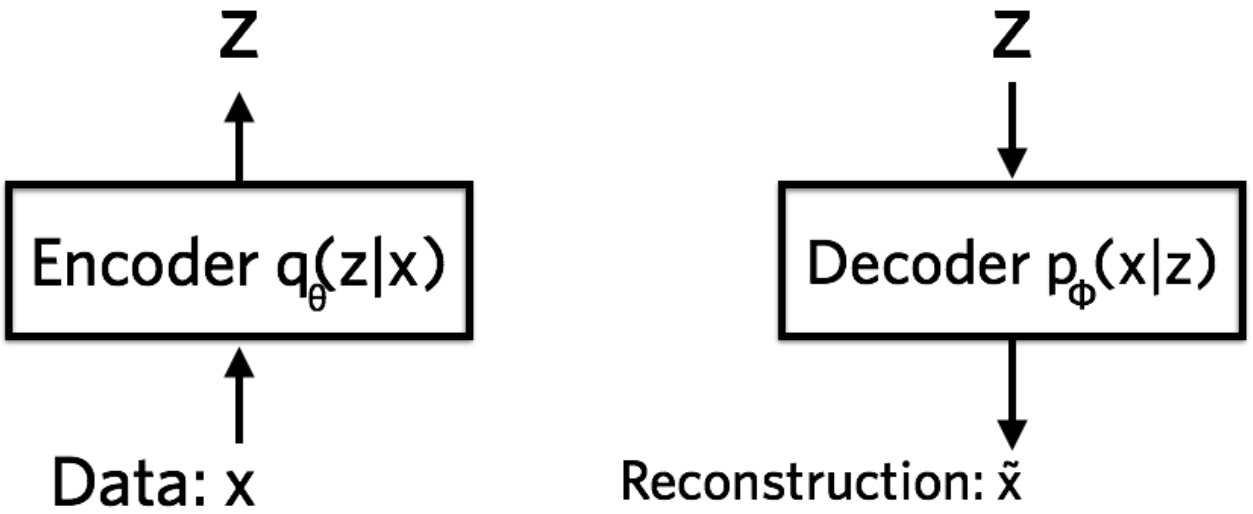


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

Regularisation term - measures how close **q** is to **p**

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

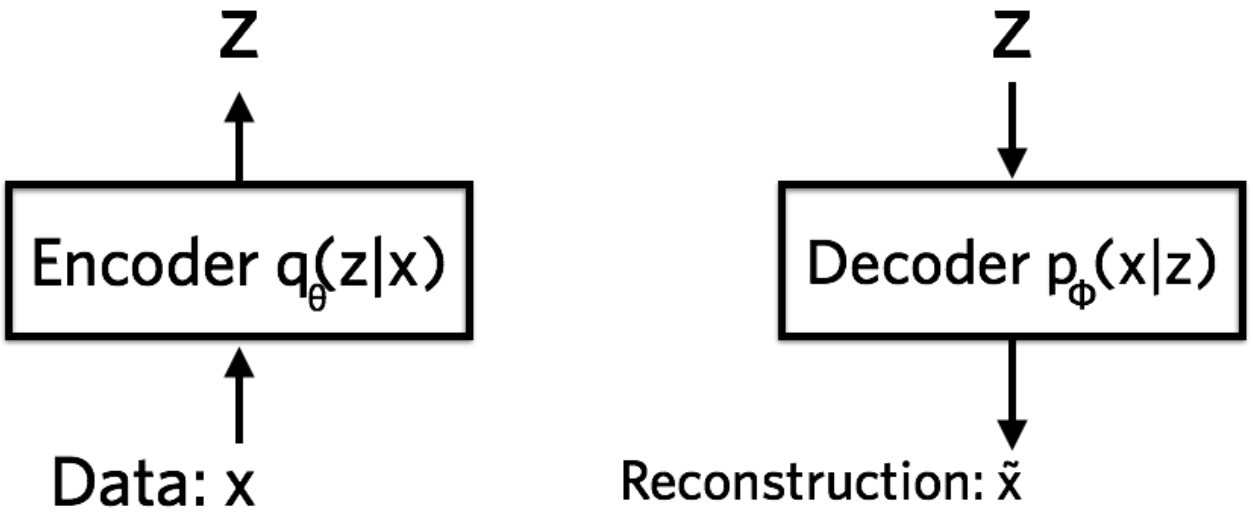


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

In VAs we let **p(z) = Gaussian(0,1)**

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

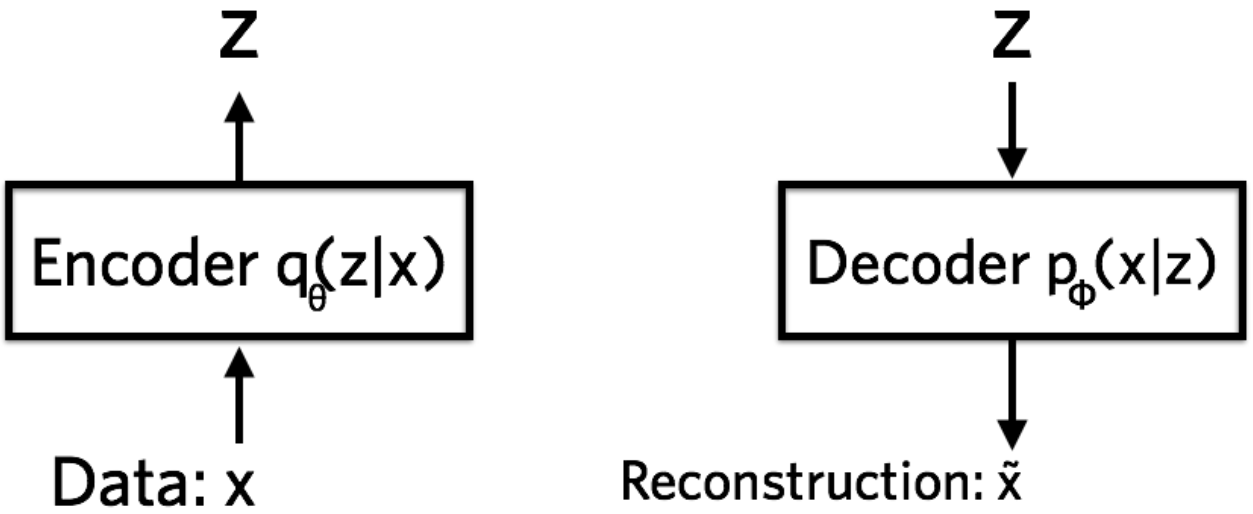


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

This is added to make sure the encoder doesn't cheat and map each datapoint in different regions of the space

VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components

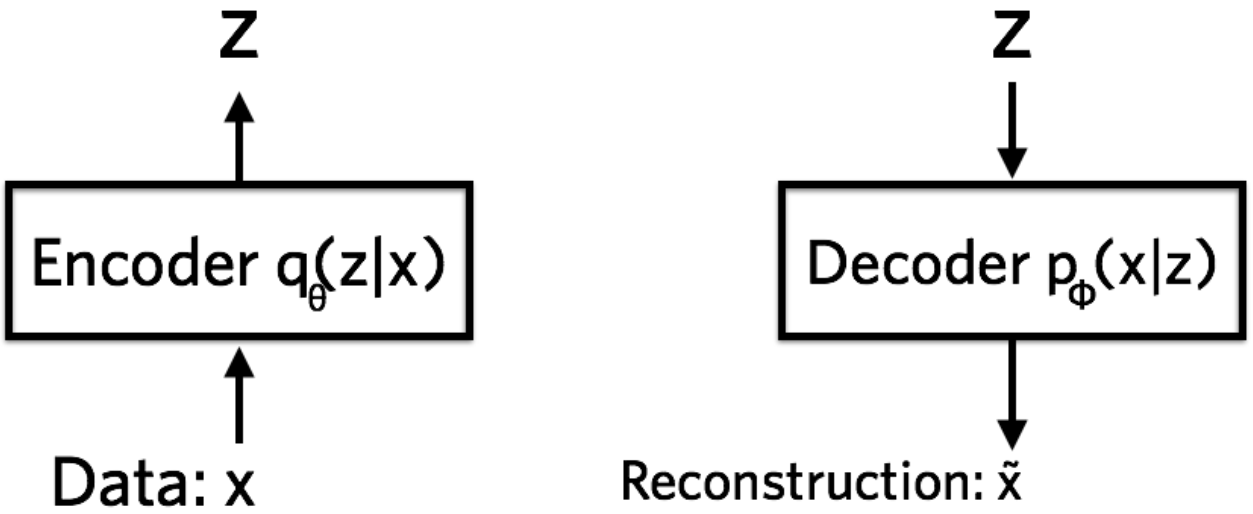


$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

This is bad because we want e.g. different images of same number to lie close in the embedding space

VAEs: neural networks perspective

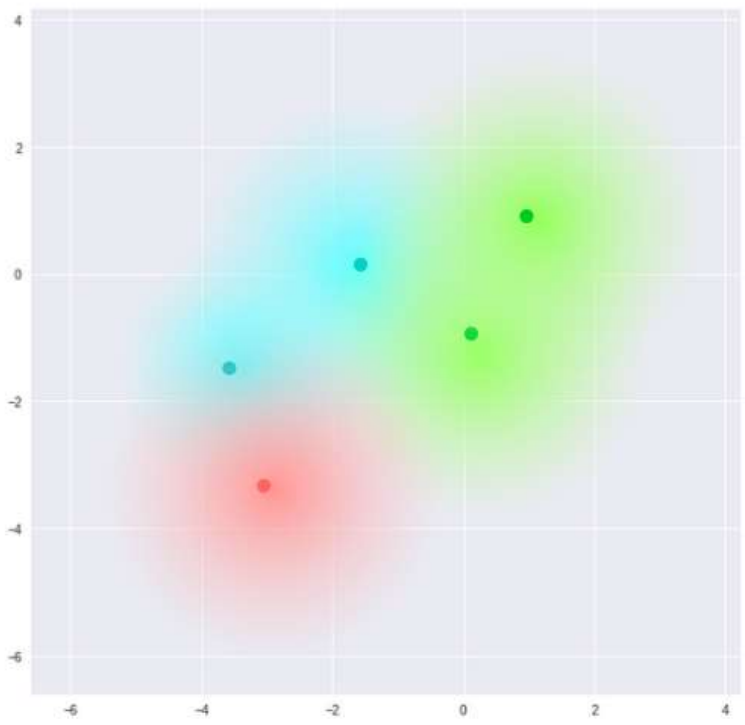
- Let's have a closer look at our (variational) autoencoder components



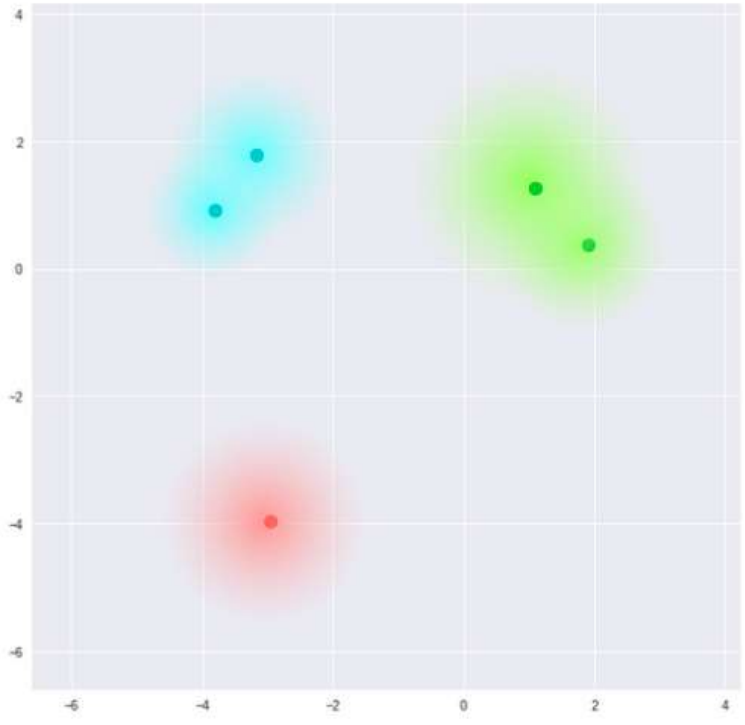
$$l_i(\theta, \phi) = -\mathbb{E}_{z \sim q_\theta(z|x_i)}[\log p_\phi(x_i \mid z)] + \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

The space has to be "meaningful", so we penalise this behaviour

Meaning of the regularisation



What we require

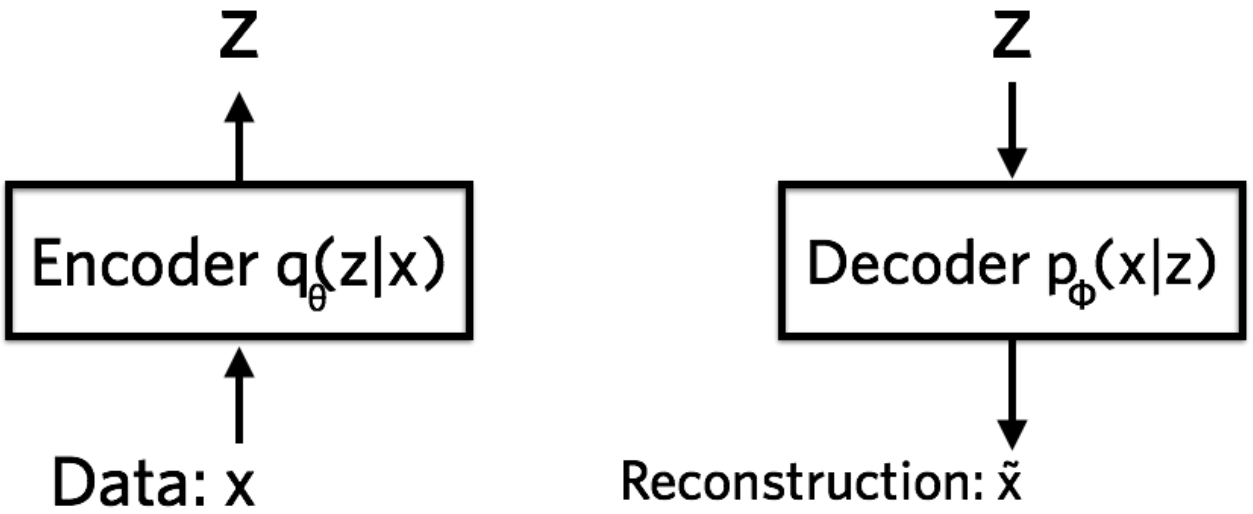


What we may inadvertently end up with

Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

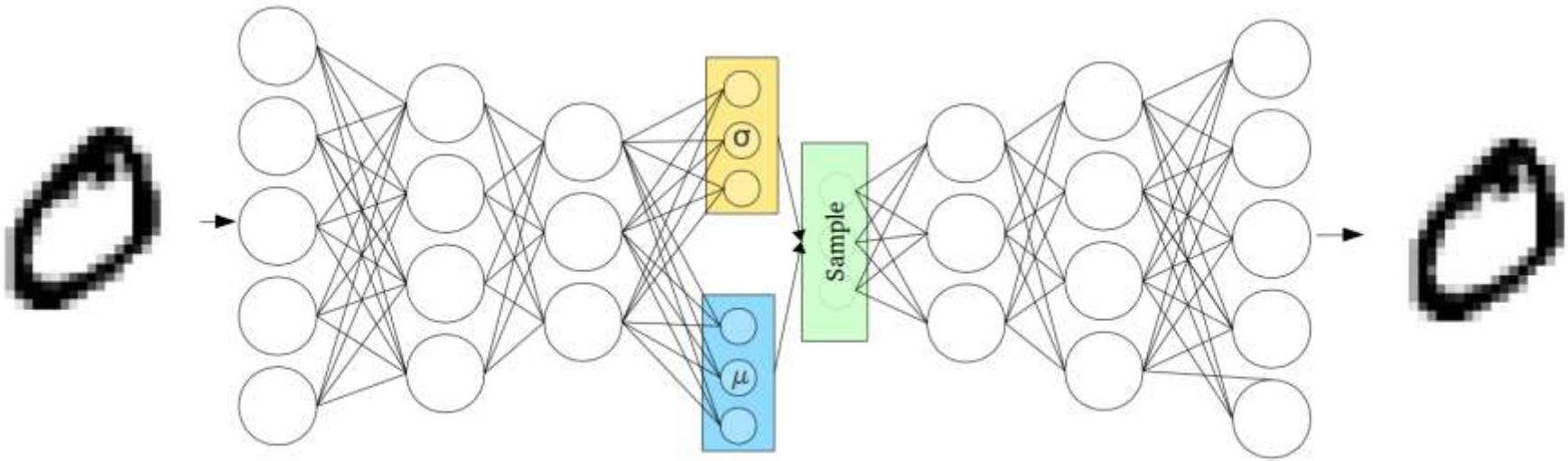
VAEs: neural networks perspective

- Let's have a closer look at our (variational) autoencoder components



Given this architecture, we can use back-propagation to compute the gradients of the loss wrt the networks parameters and then optimise using any variant of gradient descent

VAEs architecture



Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

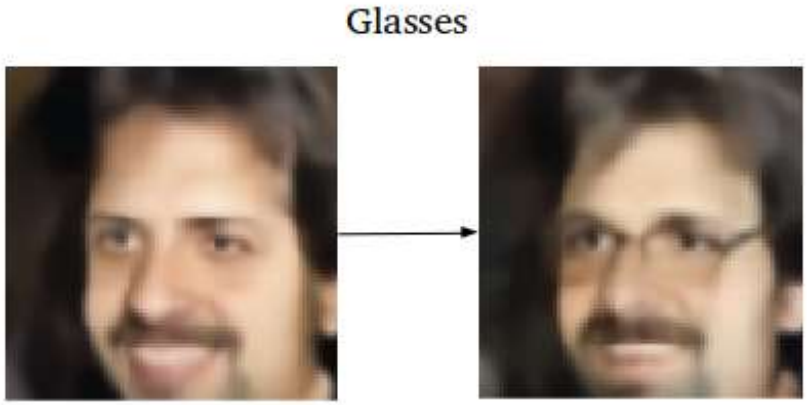
VAEs: latent space exploration

Want: find images between “face with glasses” to “face without glasses”

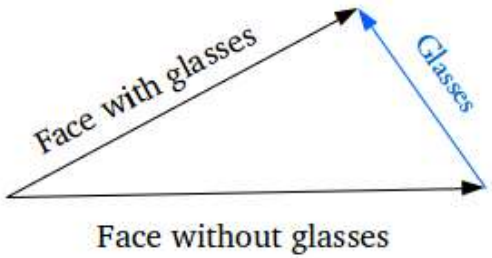
First: Find representation of faces in latent space (by giving it as input of encoder)

Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

VAEs: latent space exploration

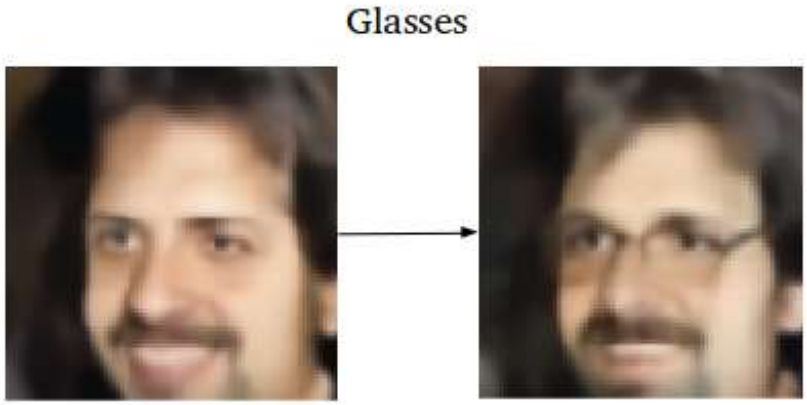


What we do: In the latent space, you computed the translation vector from “face without glasses” to “face with glasses” from the embeddings of two faces, without and with glasses

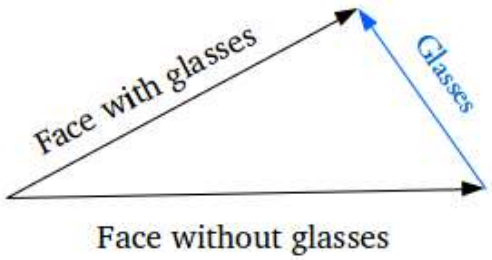


Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

VAEs: latent space exploration



Add the translation vector to the latent representation then decode this representation to map it back to the image space



Source: https://towardsdatascience.com/intuitively-understanding-variational-autoencoders-1bfe67eb5daf

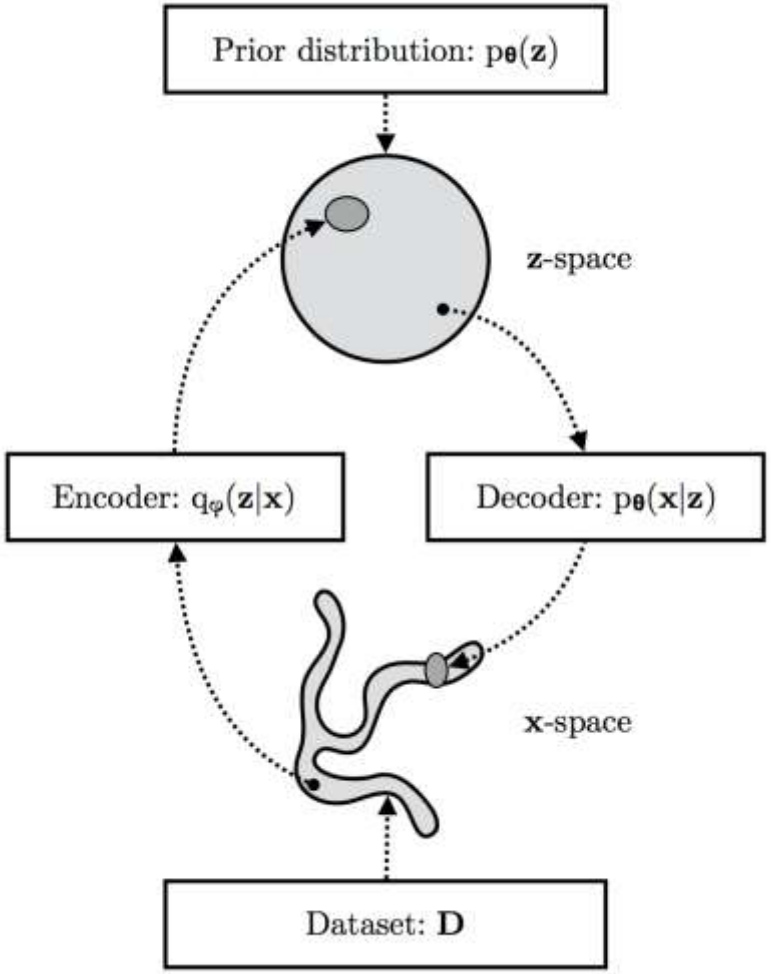
VAEs: neural networks perspective

- …but why exactly is this called a “variational” autoencoder?

VAEs: probabilistic perspective

- To understand why, we need to look at variational autoencoders from a different perspective, i.e., that of a probability model

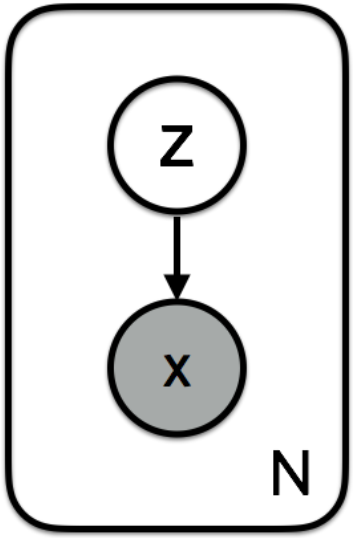
VAEs architecture



Source: D.P. Kingma Variational Inference and Deep Learning: A New Synthesis

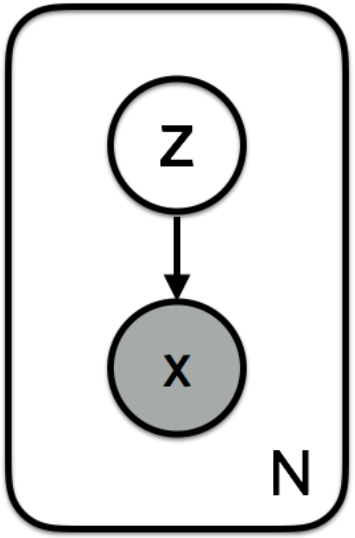
VAEs: probabilistic perspective

- To understand why, we need to look at variational autoencoders from a different perspective, i.e., that of a probability model
- Let there be a generative model of the data **x** and the latent variables **z** with joint probability **p(x,z) = p(x|z)p(z)**



VAEs: probabilistic perspective

- To understand why, we need to look at variational autoencoders from a different perspective, i.e., that of a probability model
- Let there be a generative model of the data **x** and the latent variables **z** with joint probability **p(x,z) = p(x|z)p(z)**
- First we sample **z** from prior **p(z)**
- Then we sample **x** from the likelihood **p(x|z)**



VAEs: probabilistic perspective

- In this context, learning is called *inference*
- We want to infer the optimal parameters of **p(x)**
- In other words, we want to maximise **p(x)**
- **log p(x) = log p(x,z)/p(z|x)**, to be precise
- But **p(z|x) = p(x,z)/p(x)**, and computing **p(x)** takes an exponential time since **p(x) = ∫p(x|z)p(z)dz**
- **p(z|x)** is called the posterior and its often intractable in this type of problems…

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$\mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x)) =$$

$$\mathbf{E}_q[\log q_\lambda(z \mid x)] - \mathbf{E}_q[\log p(x, z)] + \log p(x)$$

We want this to be small

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$q_\lambda^*(z \mid x) = \arg\min_\lambda \mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x))$$

This is what we're looking for

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$q_\lambda^*(z \mid x) = \arg\min_\lambda \mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x))$$

But this isn't good…

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$\mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x)) =$$

$$\mathbf{E}_q[\log q_\lambda(z \mid x)] - \mathbf{E}_q[\log p(x, z)] + \log p(x)$$

Let us introduce this function

$$ELBO(\lambda) = \mathbf{E}_q[\log p(x, z)] - \mathbf{E}_q[\log q_\lambda(z \mid x)]$$

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$\log p(x) = ELBO(\lambda) + \mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x))$$

Then we can write this

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$\log p(x) = ELBO(\lambda) + \mathbb{KL}(q_\lambda(z \mid x) \,||\, p(z \mid x))$$

KL is always >= 0, so to minimise KL we can maximise ELBO!

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **p(z|x)** with a family of distributions $\mathbf{q_\lambda(z|x)}$
- Of course we want our approximation to be good, i.e., close to the true posterior

$$\log p(x) = ELBO(\lambda) + \mathbb{KL}(q_\lambda(z \mid x) \mid\mid p(z \mid x))$$

Maximising ELBO means 1) **q** close to **p** and 2) higher **p** (better generator)

$$ELBO_i(\theta, \phi) = \mathbb{E}q_\theta(z \mid x_i)[\log p_\phi(x_i \mid z)] - \mathbb{KL}(q_\theta(z \mid x_i) \mid\mid p(z))$$

VAEs: probabilistic perspective

- Which is where the **variational** elements comes in!
- Variational inference approximates the posterior **$p(z|x)$** with a family of distributions **$q_\lambda(z|x)$**
- Of course we want our approximation to be good, i.e., close to the true posterior

$$ELBO(\lambda) = \mathbf{E}_q[\log p(x,z)] - \mathbf{E}_q[\log q_\lambda(z \mid x)]$$

$$ELBO_i(\theta,\phi) = \mathbb{E}q_\theta(z \mid x_i)[\log p_\phi(x_i \mid z)] - \mathbb{KL}(q_\theta(z \mid x_i) \,||\, p(z))$$

We link the network and probabilistic perspective by explicating the parameters of q and p and noting that the above is the (negative of the) loss function of a variational autoencoder

- Next Topic
  - Generative adversarial networks

Advancde Deep Learning

2. Generative adversarial networks *

Prof. Jianguo Zhang

SUSTech

*Based on http://www.deeplearningbook.org chapter 20.10.4 + various sources cited in the slides

Generative tasks

- Generation (from scratch): learn to sample from the distribution represented by the training set
  - *Unsupervised learning* task

Generative tasks

- Conditional generation

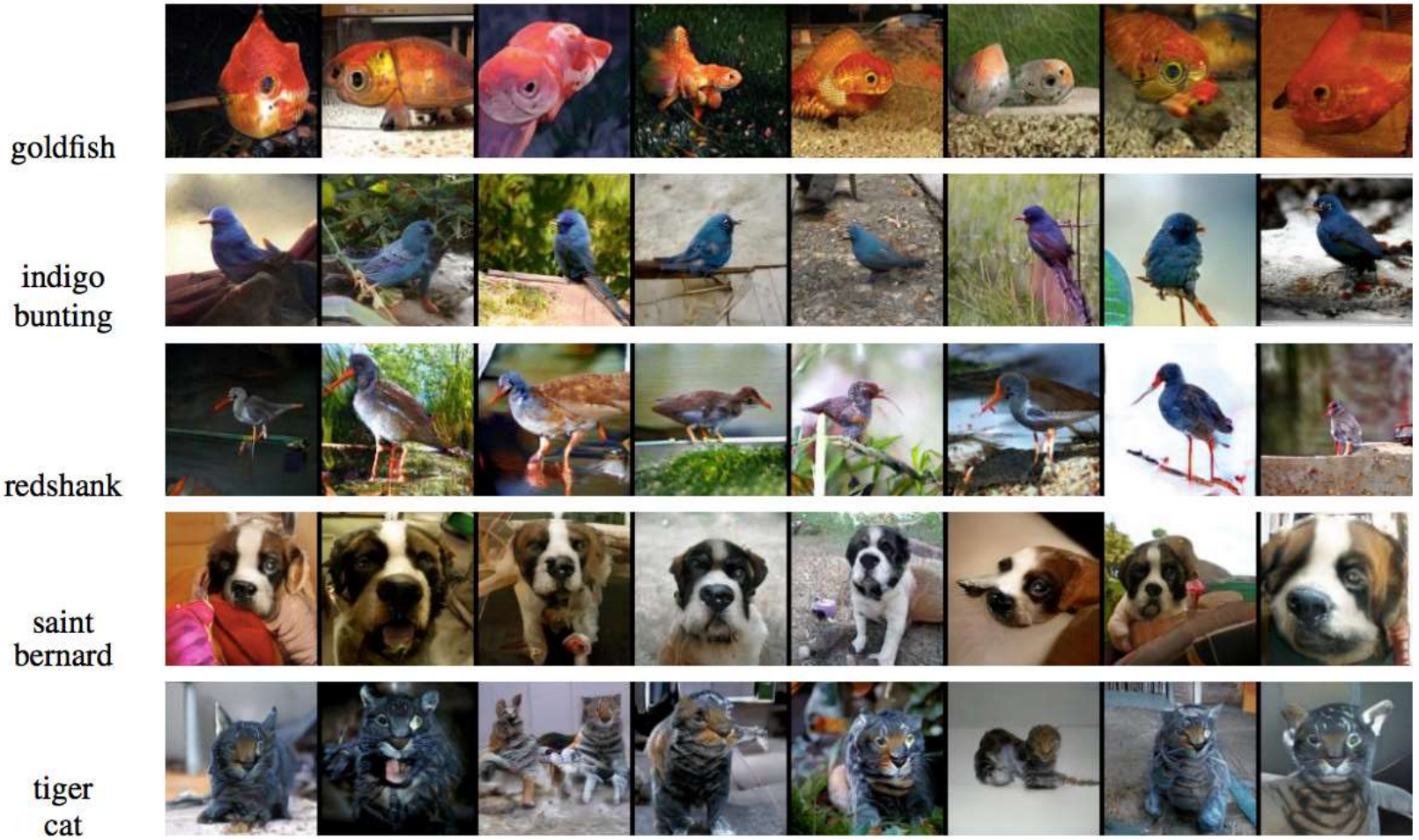
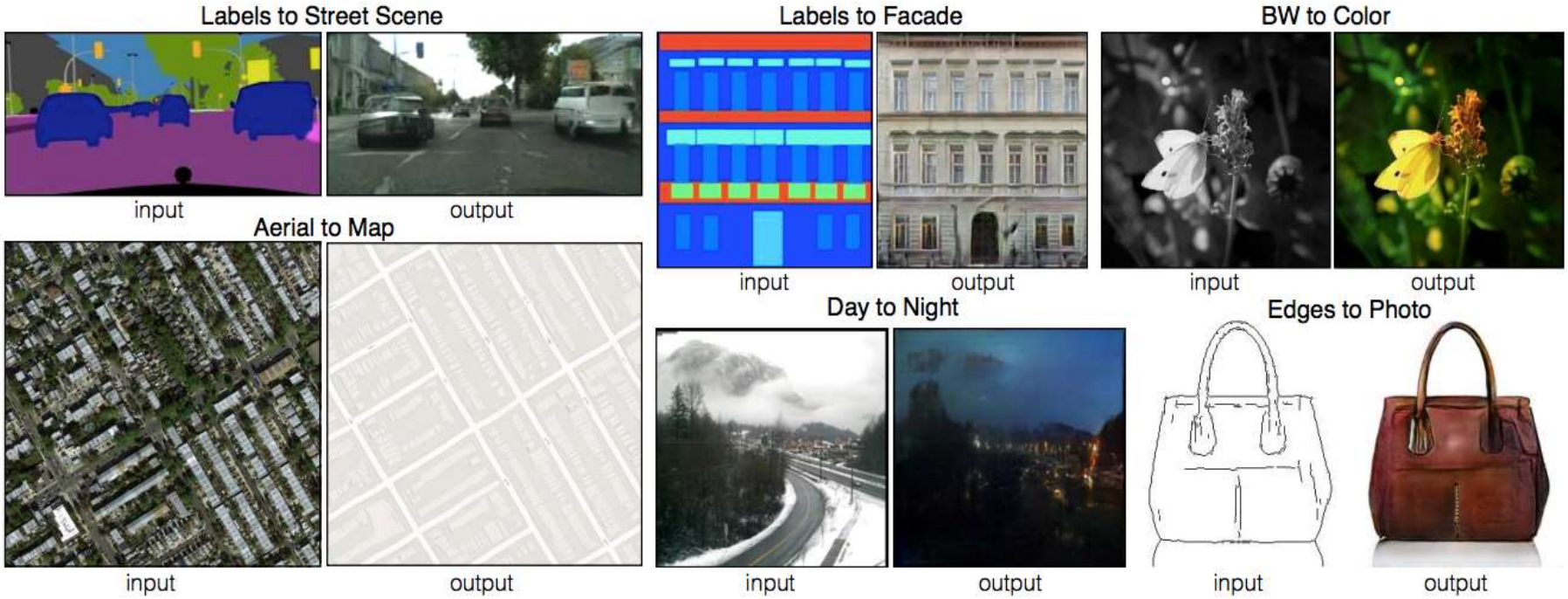


Figure source

Generative tasks

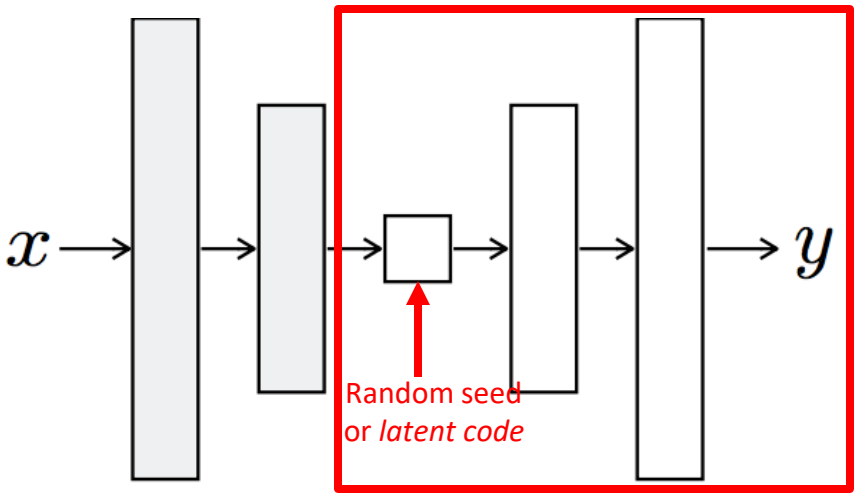
- Image-to-image translation



P. Isola, J.-Y. Zhu, T. Zhou, A. Efros, Image-to-Image Translation with Conditional Adversarial Networks, CVPR 2017

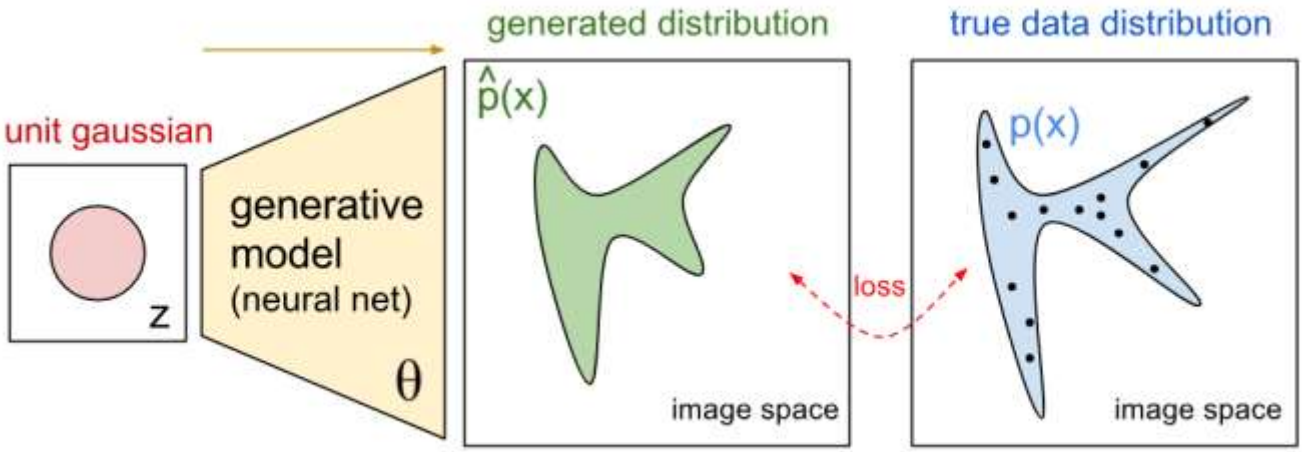
Designing a network for generative tasks

- We saw that VAEs can learn to generate data by training both an encoder and a decoder (generator)
- Can we learn just the generator?
  - Recall the decoder of VAE



From VAEs to GANs

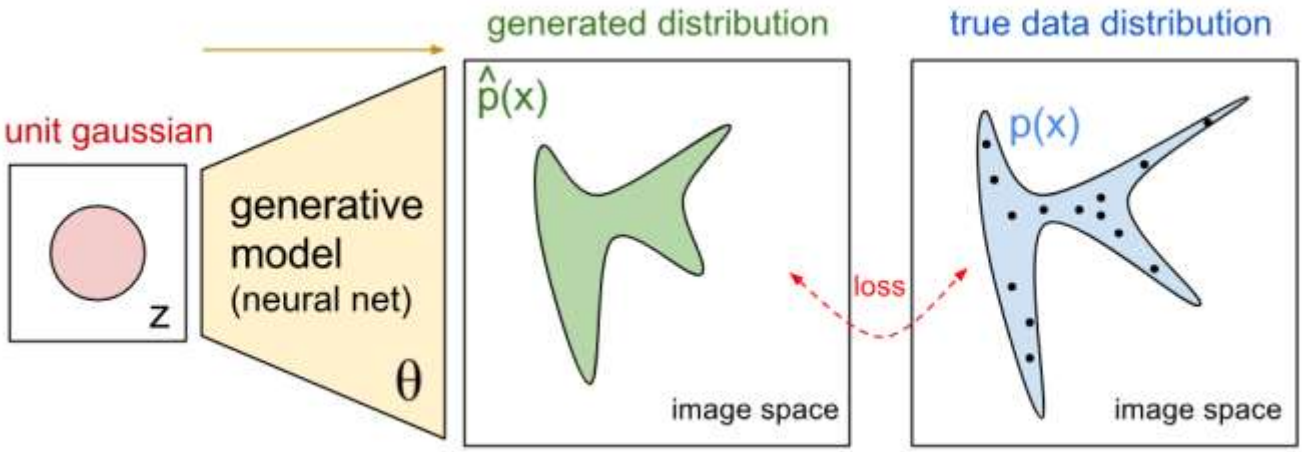
- We saw that VAEs can learn to generate data by training both an encoder and a decoder (generator)
- Can we learn just the generator?



Source: https://openai.com/blog/generative-models/

Generative adversarial networks

- GANs propose to learn the loss function
- The training process is a game between two networks



Source: https://openai.com/blog/generative-models/

Generative adversarial networks

- Train two networks with opposing objectives:
  - **Generator:** learns to generate samples
  - **Discriminator:** learns to distinguish between generated and real samples

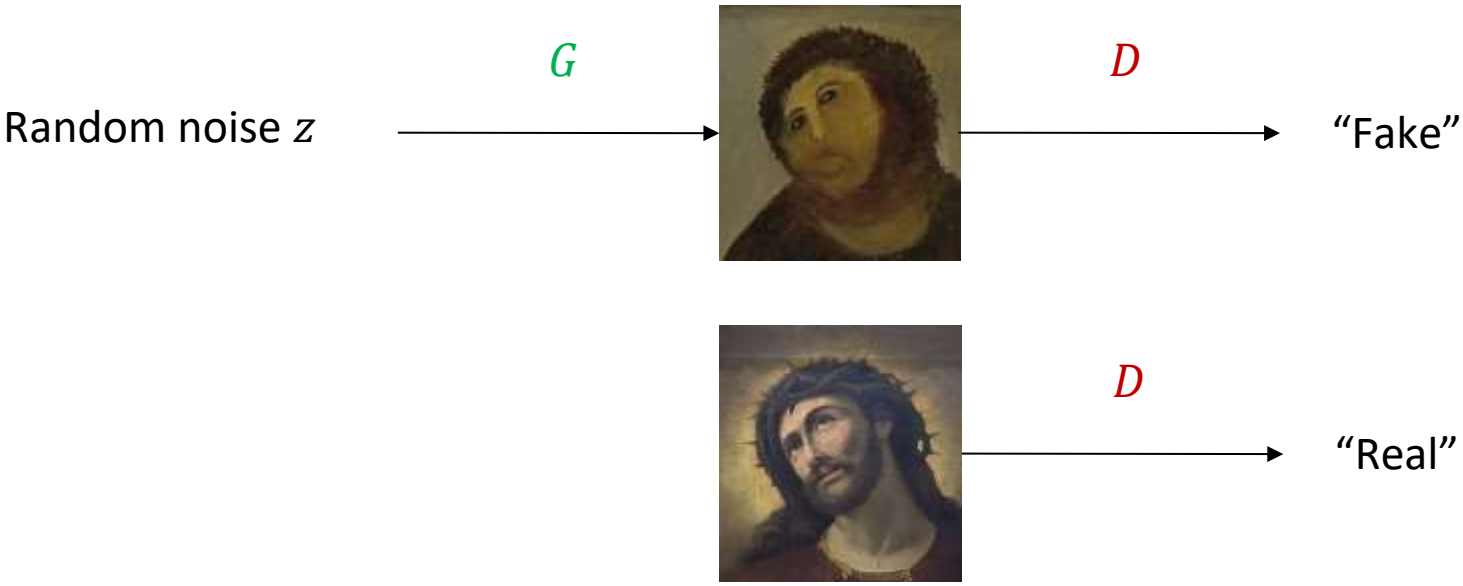
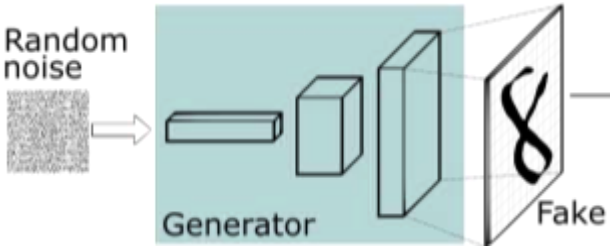


Figure adapted from F. Fleuret

I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, Y. Bengio, Generative adversarial nets, NIPS 2014

Generative adversarial networks

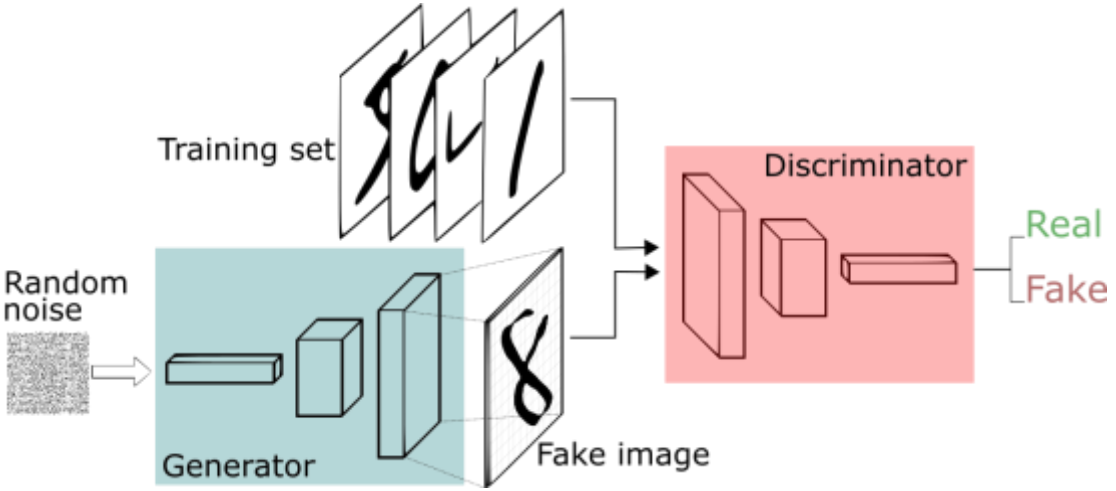
- **Generator**: G(z) takes random noise z as input and outputs a (fake) image



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Generative adversarial networks

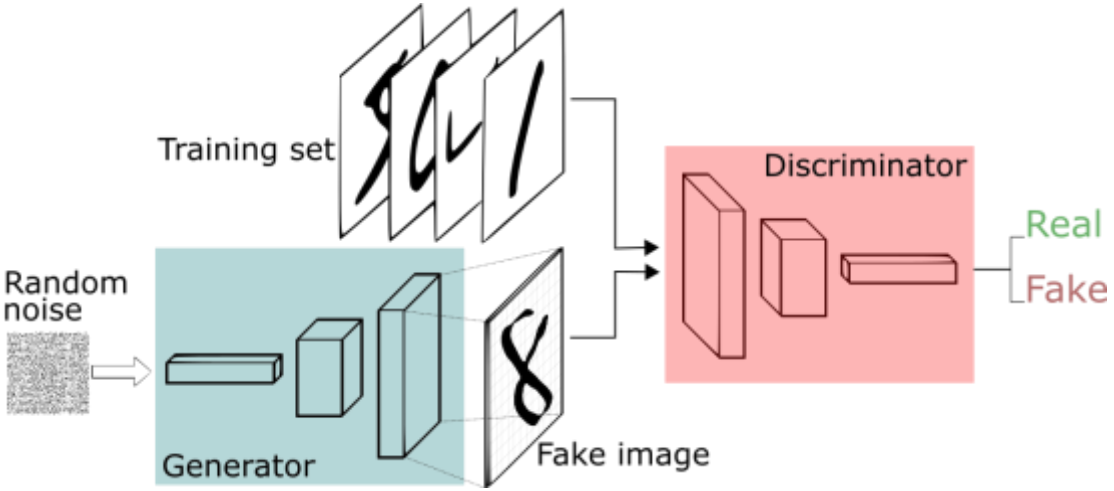
- **Generator**: G(z) takes random noise z as input and outputs a (fake) image
- **Discriminator**: D(x) receives an image x in input, real or fake, and estimates its probability to be real



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Generative adversarial networks

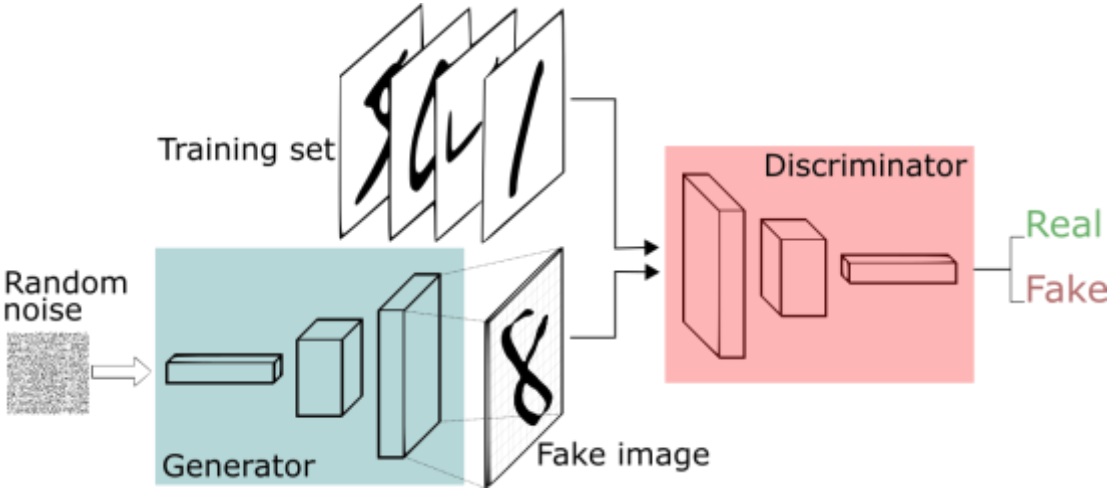
- **Adversarial training**: the generator tries to fool the discriminator while the discriminator tries to get better at distinguishing fake vs real images



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Generative adversarial networks

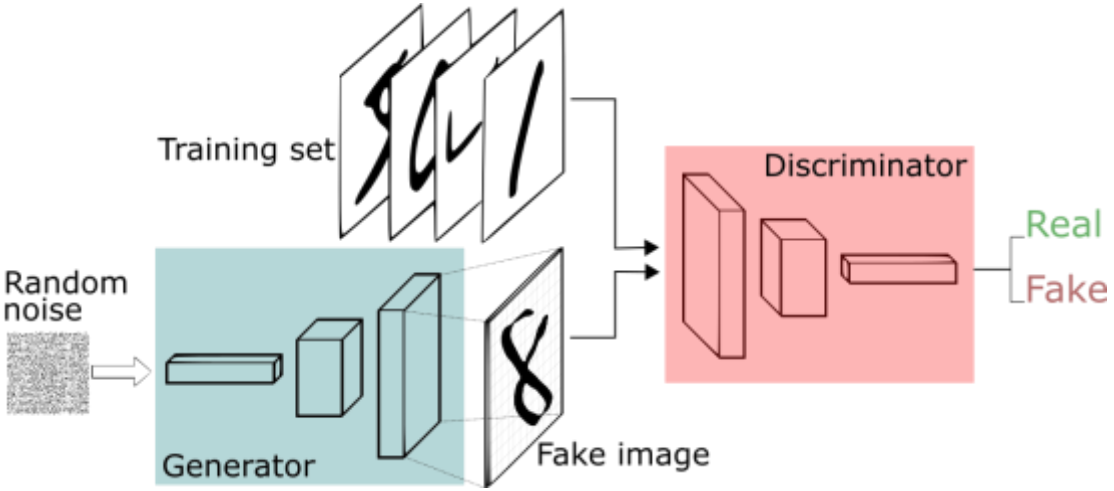
- When the discriminator spots a fake the generator adjusts its parameters, until at the end the generator reproduces the true data distribution and the discriminator is unable to find differences



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Generative adversarial networks

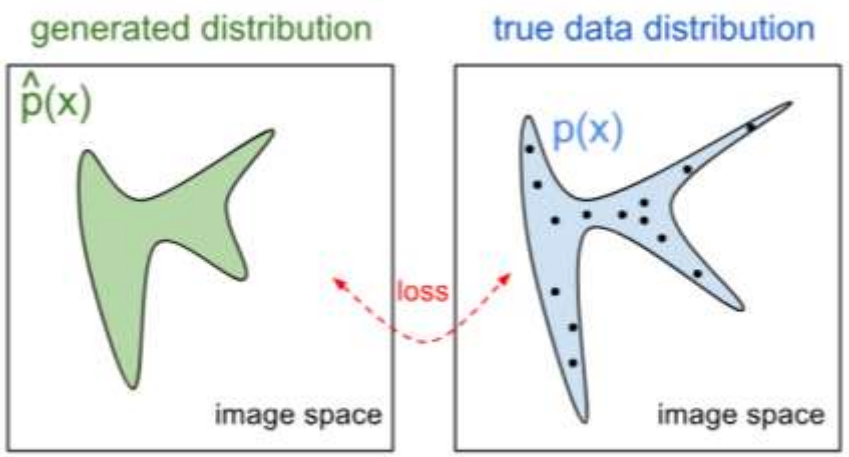
- **Note**: both the generator and the discriminator need to be differentiable
- Typically both implemented as (deep) neural **networks**, so we can use backpropagation



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

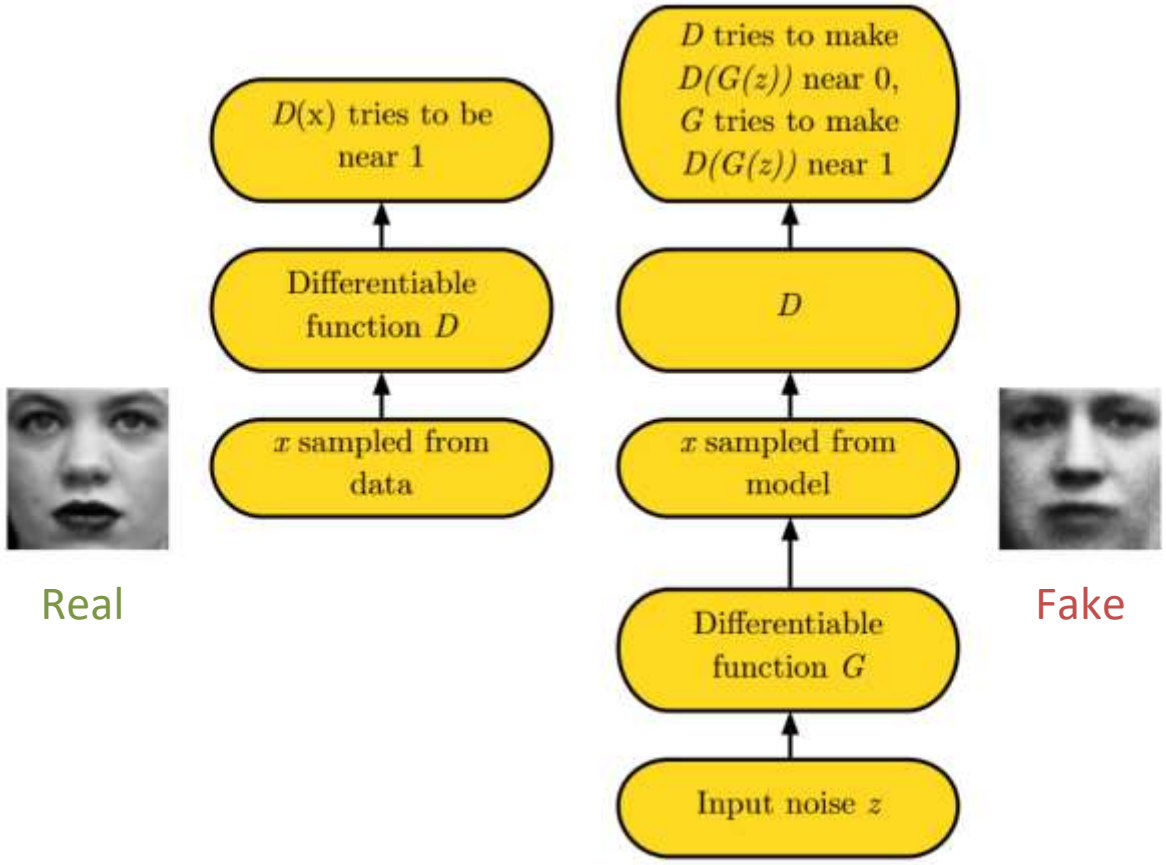
Generative adversarial networks

- That's like saying that the discriminator at the end in unable to tell the difference between the generated data distribution and the true data distribution (where training data comes from)

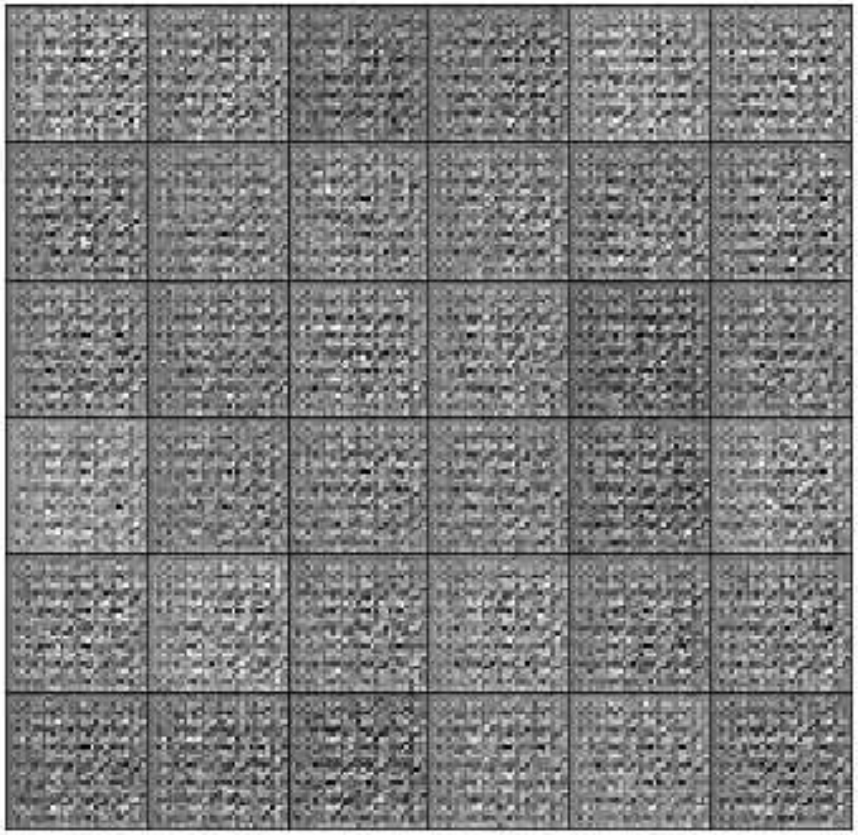


Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

GANs pipeline



Source: UVA 2018 deep learning course



Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Source: https://medium.freecodecamp.org/an-intuitive-introduction-to-generative-adversarial-networks-gans-7a2264a81394

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Prior distribution on input noise variables

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

The generator is a differentiable function (e.g., MLP) mapping noise to data space

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

The discriminator is a differentiable function (e.g., MLP) with single scalar output, i.e., the probability that *x* comes from the true data distribution

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

D(*x*) = 1 when the discriminator thinks *x* is a real image

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

D(G(*z*)) = 1 when the discriminator thinks G(*z*) is a real image

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

D is trained to maximise the probability of giving the correct label to samples from the true data distribution (the label should be 1) as well as to samples from G (the label should be 0)

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Large when D(*x*) close to 1

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Large when D(G(*z*)) close to 0

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

Simultaneously G is trained to minimise log(1 − D(G(*z*))), i.e., fool the discriminator

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

- For a fixed G, the loss is effectively the binary cross-entropy
- The minimax strategy is used for **zero-sum games**, i.e., loss of one player = gain of the adversary
- The minimax solution is the **Nash equilibrium**

Nash equilibrium is a state in which D and G don't have any incentive to change because doing so would make the value of the objective function worse

Minimax and zero-sum games

- G and D follow a **minimax strategy**

$$\min_G \max_D V(D,G) = \mathbb{E}_{\boldsymbol{x}\sim p_{\text{data}}(\boldsymbol{x})}[\log D(\boldsymbol{x})] + \mathbb{E}_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}[\log(1 - D(G(\boldsymbol{z})))]$$

- For a fixed G, the loss is effectively the binary cross-entropy
- The minimax strategy is used for **zero-sum games**, i.e., loss of one player = gain of the adversary
- The minimax solution is the **Nash equilibrium**

In machine learning terms it means we reached convergence of the gradient descent for the joint optimisation problem

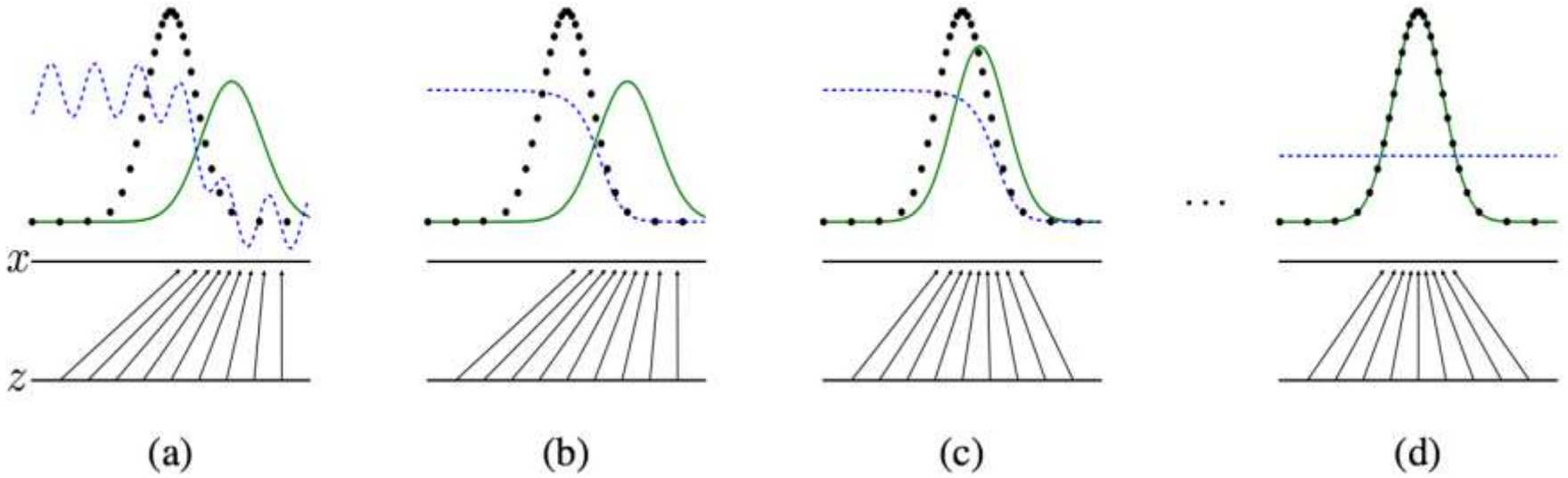


Figure 1: Generative adversarial nets are trained by simultaneously updating the **d**iscriminative distribution (D, blue, dashed line) so that it discriminates between samples from the data generating distribution (black, dotted line) $p_{\boldsymbol{x}}$ from those of the **g**enerative distribution p_g (G) (green, solid line). The lower horizontal line is the domain from which $\boldsymbol{z}$ is sampled, in this case uniformly. The horizontal line above is part of the domain of $\boldsymbol{x}$. The upward arrows show how the mapping $\boldsymbol{x} = G(\boldsymbol{z})$ imposes the non-uniform distribution p_g on transformed samples. G contracts in regions of high density and expands in regions of low density of p_g. (a) Consider an adversarial pair near convergence: p_g is similar to p_{data} and D is a partially accurate classifier. (b) In the inner loop of the algorithm D is trained to discriminate samples from data, converging to $D^*(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$. (c) After an update to G, gradient of D has guided $G(\boldsymbol{z})$ to flow to regions that are more likely to be classified as data. (d) After several steps of training, if G and D have enough capacity, they will reach a point at which both cannot improve because $p_g = p_{\text{data}}$. The discriminator is unable to differentiate between the two distributions, i.e. $D(\boldsymbol{x}) = \frac{1}{2}$.

Source: https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf

GAN training

•$V(G,D) = \mathbb{E}_{x\sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z\sim p} \log(1 - D(G(z)))$

- Alternate between
  - *Gradient ascent* on discriminator:

$$D^* = \arg\max_D V(G,D)$$

  - *Gradient descent* on generator (minimize log-probability of discriminator being right):

$$G^* = \arg\min_G V(G,D)$$
$$= \arg\min_G \mathbb{E}_{z\sim p} \log(1 - D(G(z)))$$

  - In practice, do *gradient ascent* on generator (maximize log-probability of discriminator being wrong):

$$G^* = \arg\max_G \mathbb{E}_{z\sim p} \log(D(G(z)))$$

Training GANs

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Source: https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf

Training GANs

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right).$$

In practice this saturates in the beginning since D rejects the samples with high confidence because they are clearly different from the training data

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Source: https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf

Training GANs

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k, is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

  for k steps **do**

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
  - Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D\left(\boldsymbol{x}^{(i)}\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right) \right].$$

  end for

  - Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
  - Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(D\left(G\left(z^{(i)}\right)\right)\right)$$

So we use this loss function for the generator instead, i.e., the generator maximises the log-prob of the discriminator being mistaken

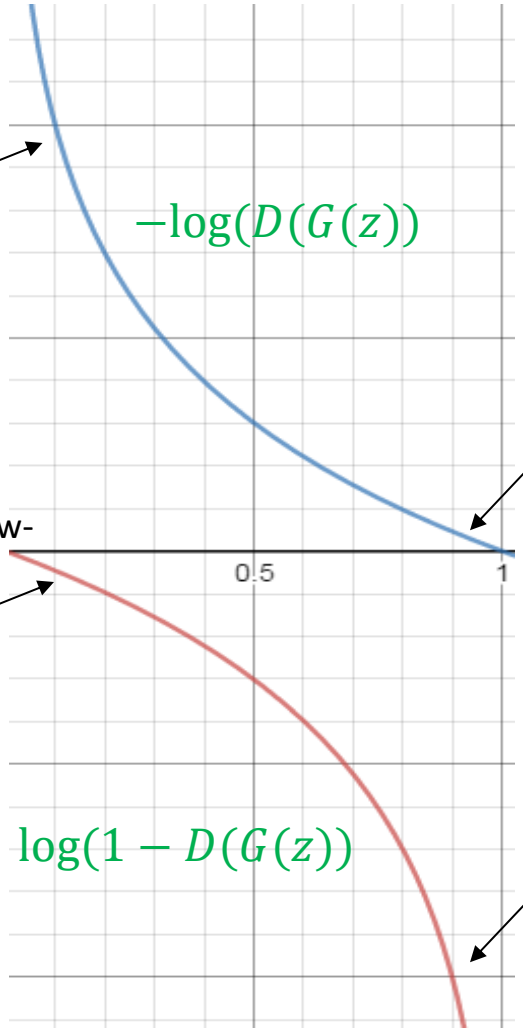
end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Source: https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf

GAN training

$\min_{w_G} \mathbb{E}_{z \sim p} \log(1 - D(G(z)))$ vs. $\max_{w_G} \mathbb{E}_{z \sim p} \log(D(G(z)))$



Large gradients for low-quality samples

Small gradients for high-quality samples

Low discriminator score (low-quality samples)

High discriminator score (high-quality samples)

Want to learn from confidently rejected sample but gradients here are small

These samples already fool the discriminator so we don't need large gradients here

Figure source

Example generator: DCGAN

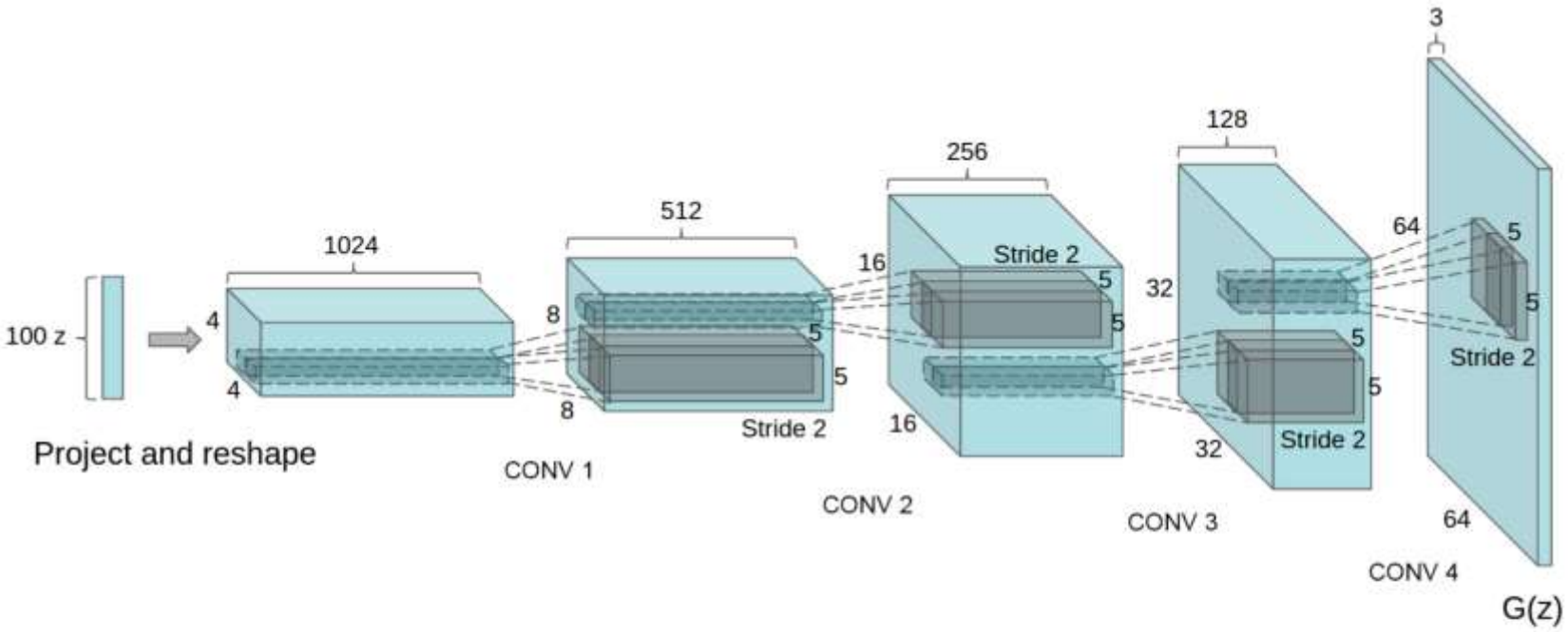


Figure 1: DCGAN generator used for LSUN scene modeling. A 100 dimensional uniform distribution Z is projected to a small spatial extent convolutional representation with many feature maps. A series of four fractionally-strided convolutions (in some recent papers, these are wrongly called deconvolutions) then convert this high level representation into a 64×64 pixel image. Notably, no fully connected or pooling layers are used.

Source: https://arxiv.org/pdf/1511.06434.pdf

DCGAN results

- Pose transformation by adding a “turn” vector

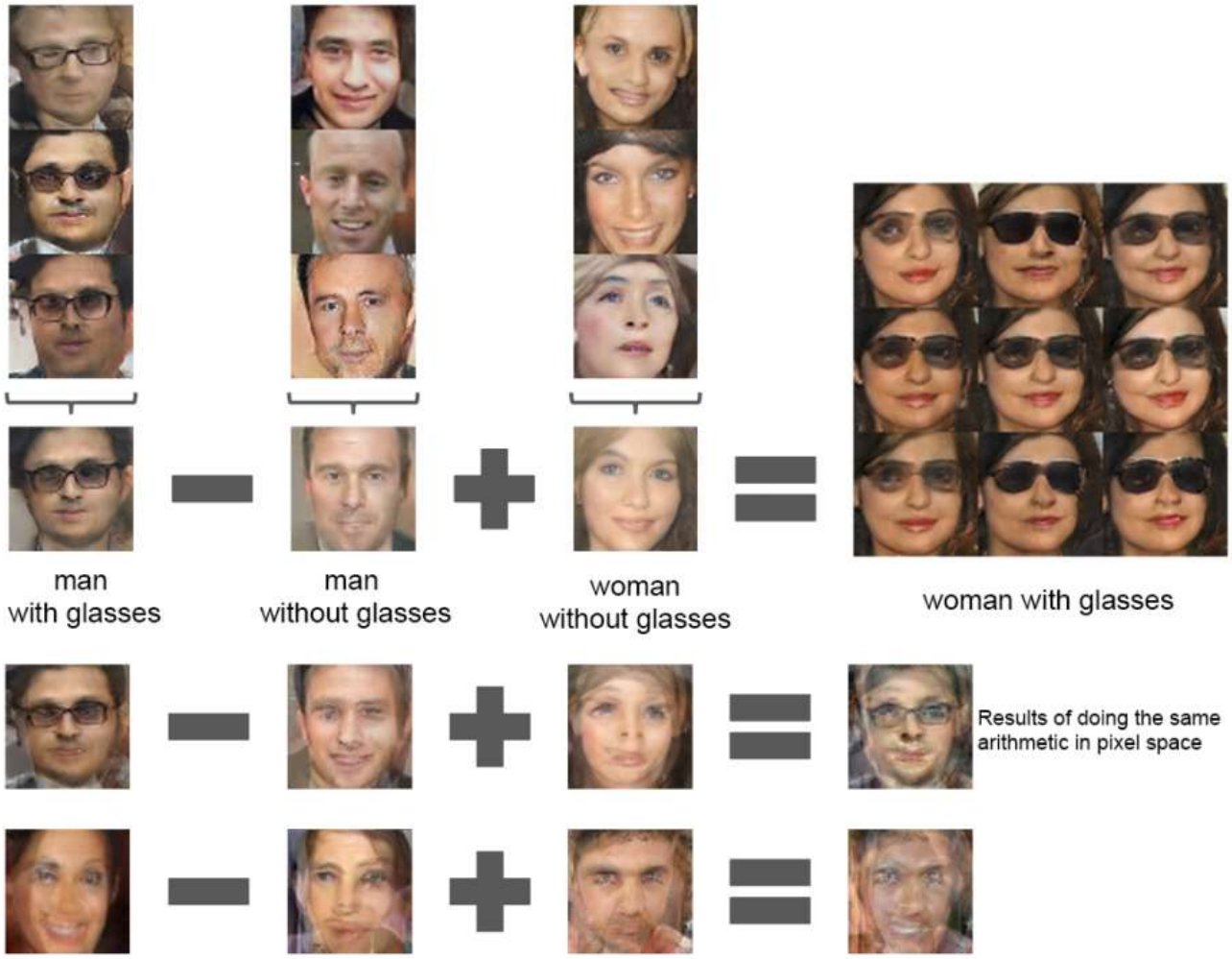


Figure 7: Vector arithmetic for visual concepts. For each column, the Z vectors of samples are averaged. Arithmetic was then performed on the mean vectors creating a new vector Y. The center sample on the right hand side is produce by feeding Y as input to the generator. To demonstrate the interpolation capabilities of the generator, uniform noise sampled with scale +-0.25 was added to Y to produce the 8 other samples. Applying arithmetic in the input space (bottom two examples) results in noisy overlap due to misalignment.

Source: https://arxiv.org/pdf/1511.06434.pdf

Problems with GANs

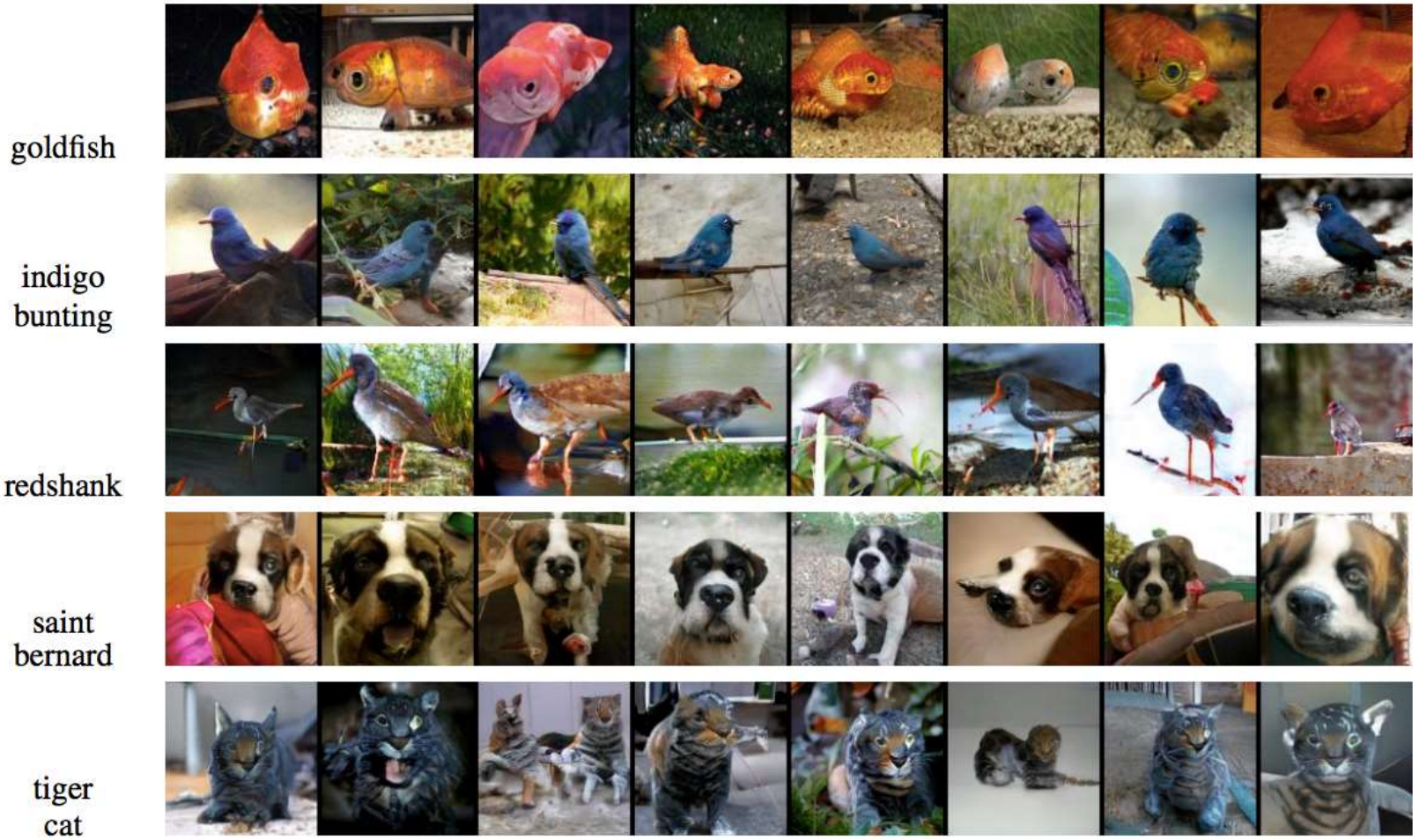
- In the real world we face multiple problems when training GANs:
  - Finite sample size: training set is finite, not the full distribution
  - Limited capacity: the generator has limit capacity, i.e. cannot perfectly represent any distribution
  - Optimisation errors: optimisers can get stuck in local optima or never exactly converge to global optima

Problems with GANs

- In the real world we face multiple problems when training GANs:
  - Saddle point problem: harder than finding a maximum or minimum
  - Balancing updates: D too weak/strong means no gradient for G to improve

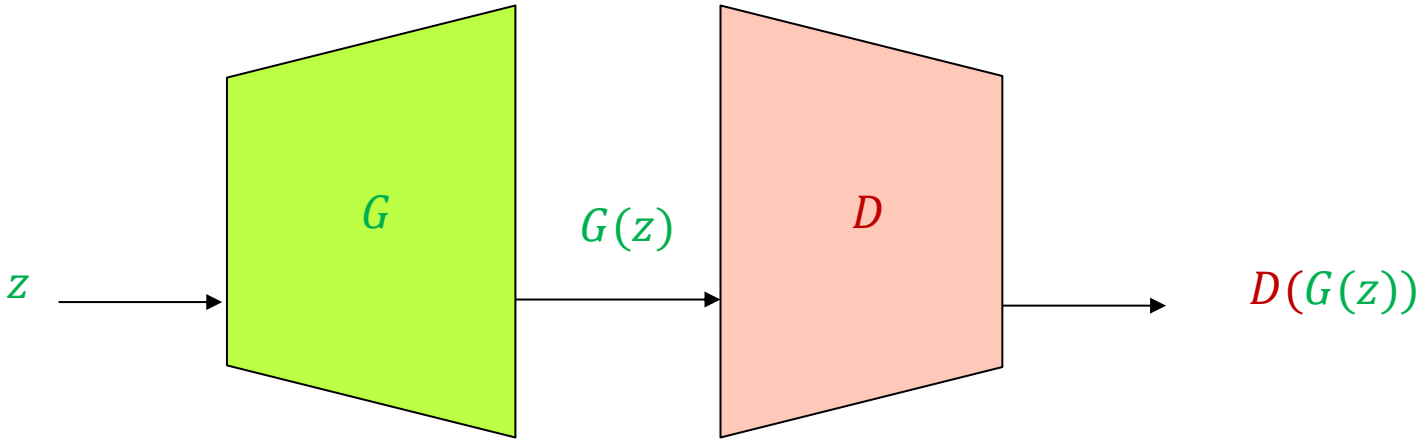
M. Arjovsky, S. Chintala, L. Bottou, Wasserstein generative adversarial networks

Conditional generation (cGAN)



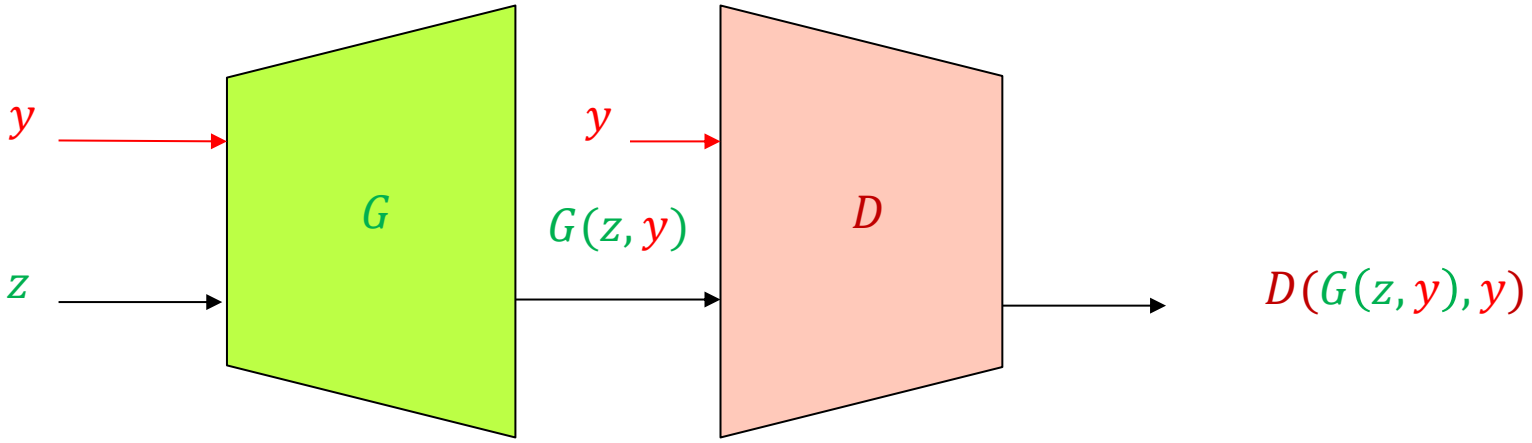
Conditional generation (cGAN)

- Suppose we want to condition the generation of samples on discrete side information (label) y
  - How do we add y to the basic GAN framework?



Conditional generation

- Suppose we want to condition the generation of samples on discrete side information (label) y
  - How do we add y to the basic GAN framework?



Conditional generation

- Example: simple network for generating 28 x 28 MNIST digits

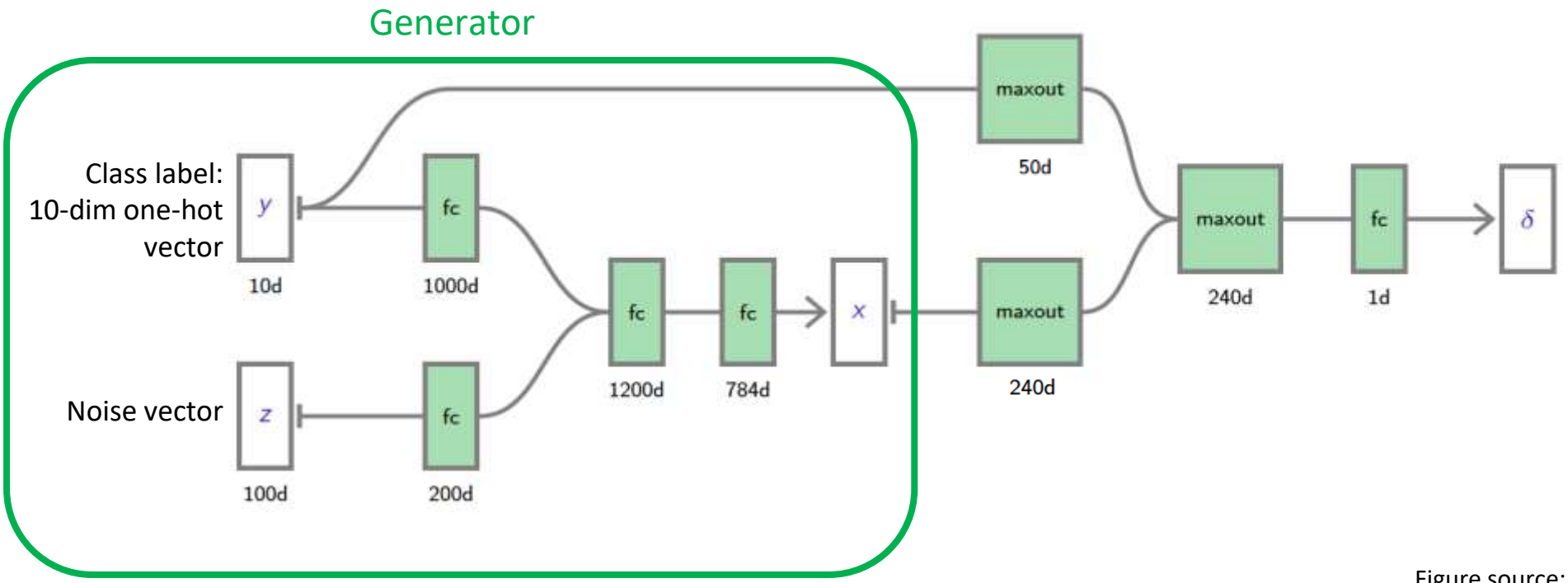


Figure source:
F. Fleuret

M. Mirza and S. Osindero, Conditional Generative Adversarial Nets, arXiv 2014

Conditional generation

- Example: simple network for generating 28 x 28 MNIST digits

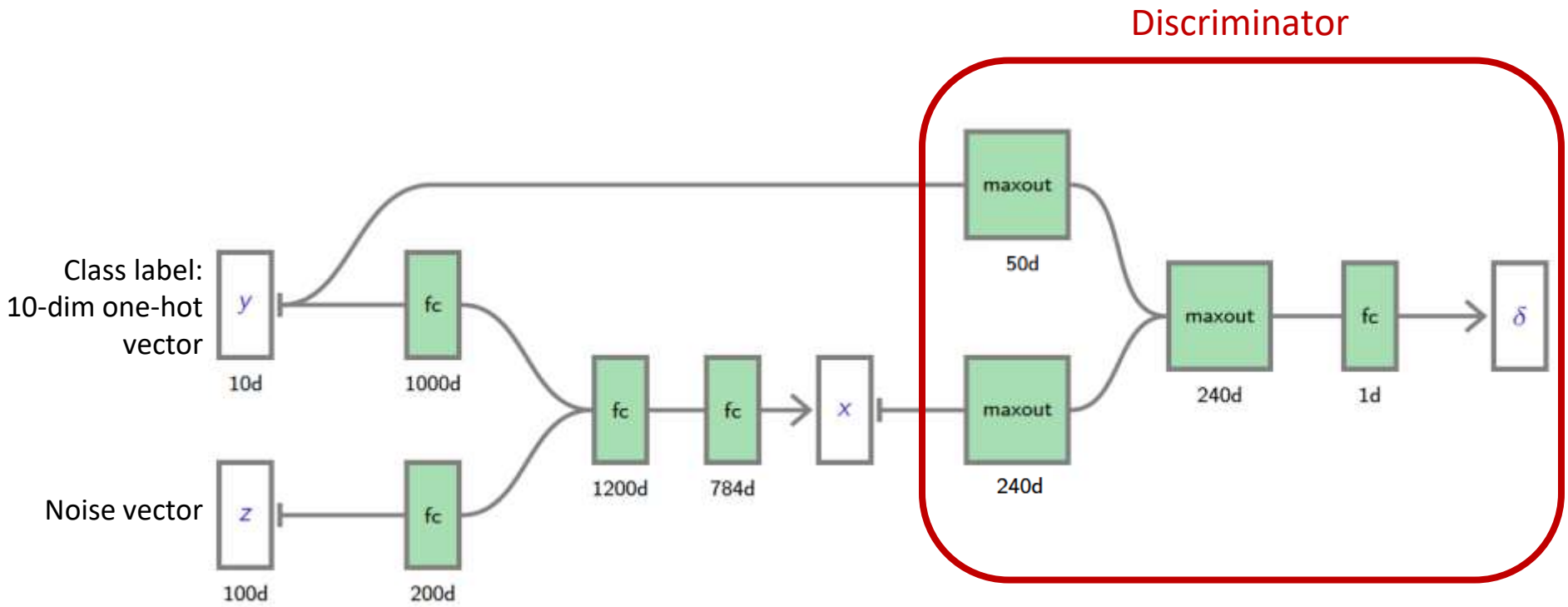
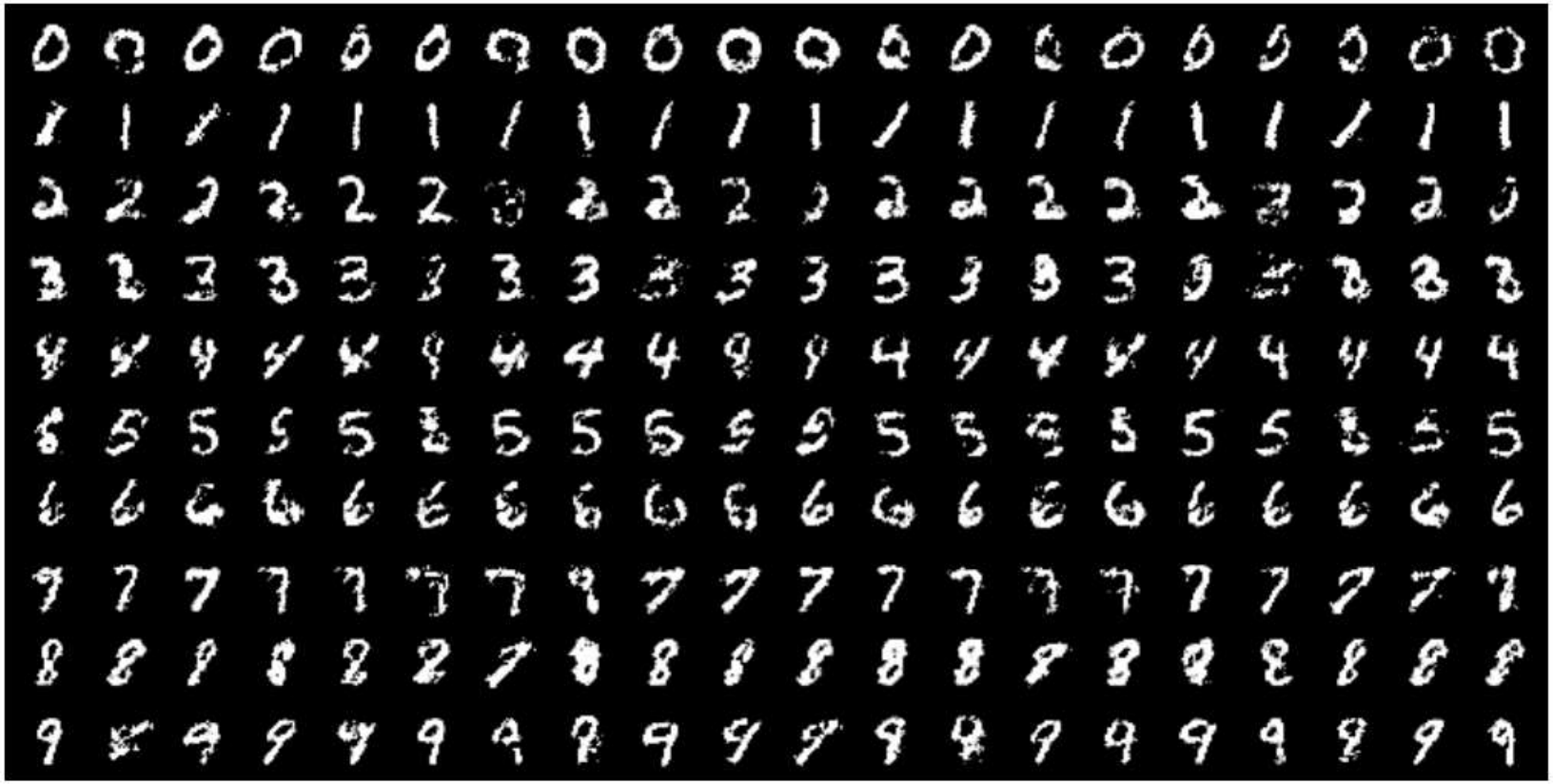


Figure source:
F. Fleuret

M. Mirza and S. Osindero, Conditional Generative Adversarial Nets, arXiv 2014

Conditional generation

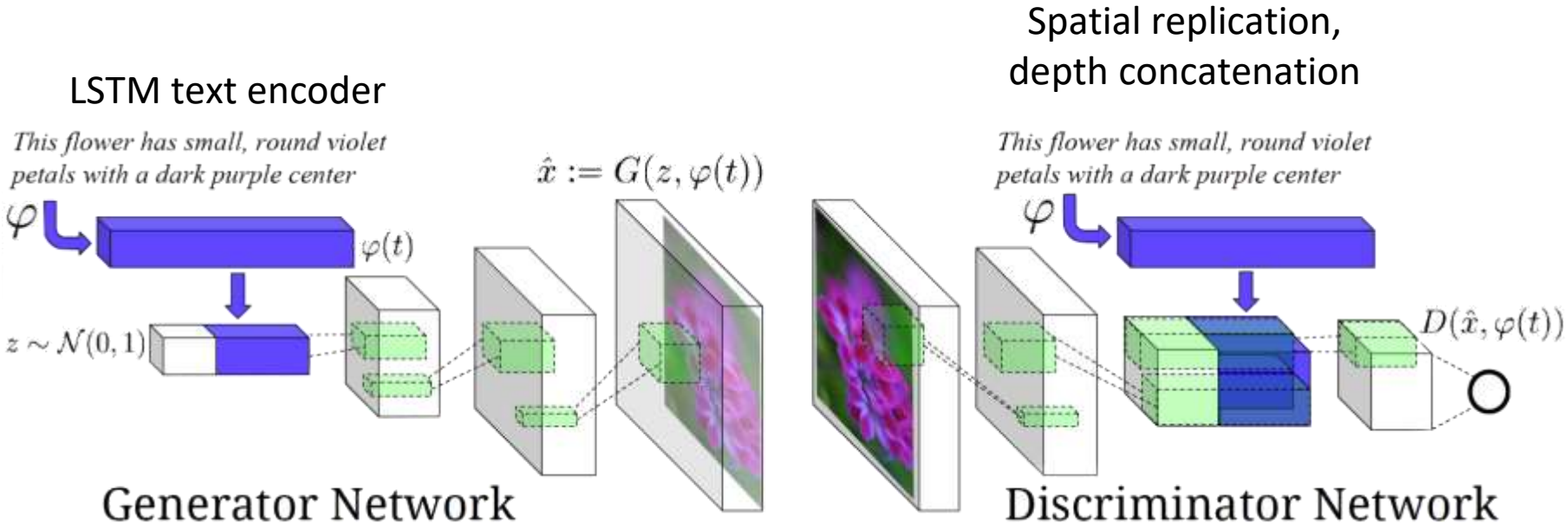
- Example: simple network for generating 28 x 28 MNIST digits



M. Mirza and S. Osindero, Conditional Generative Adversarial Nets, arXiv 2014

Conditional generation

- Another example: text-to-image synthesis

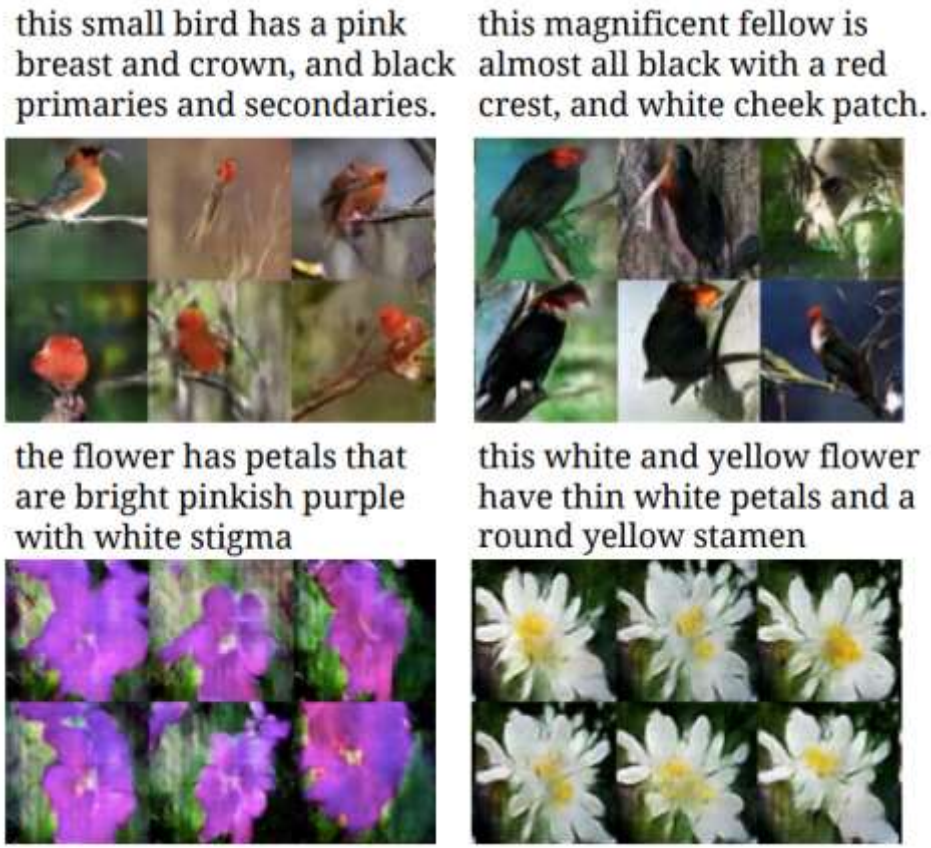


S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, H. Lee, Generative adversarial text to image synthesis, ICML 2016

Conditional generation

- Another example: text-to-image synthesis

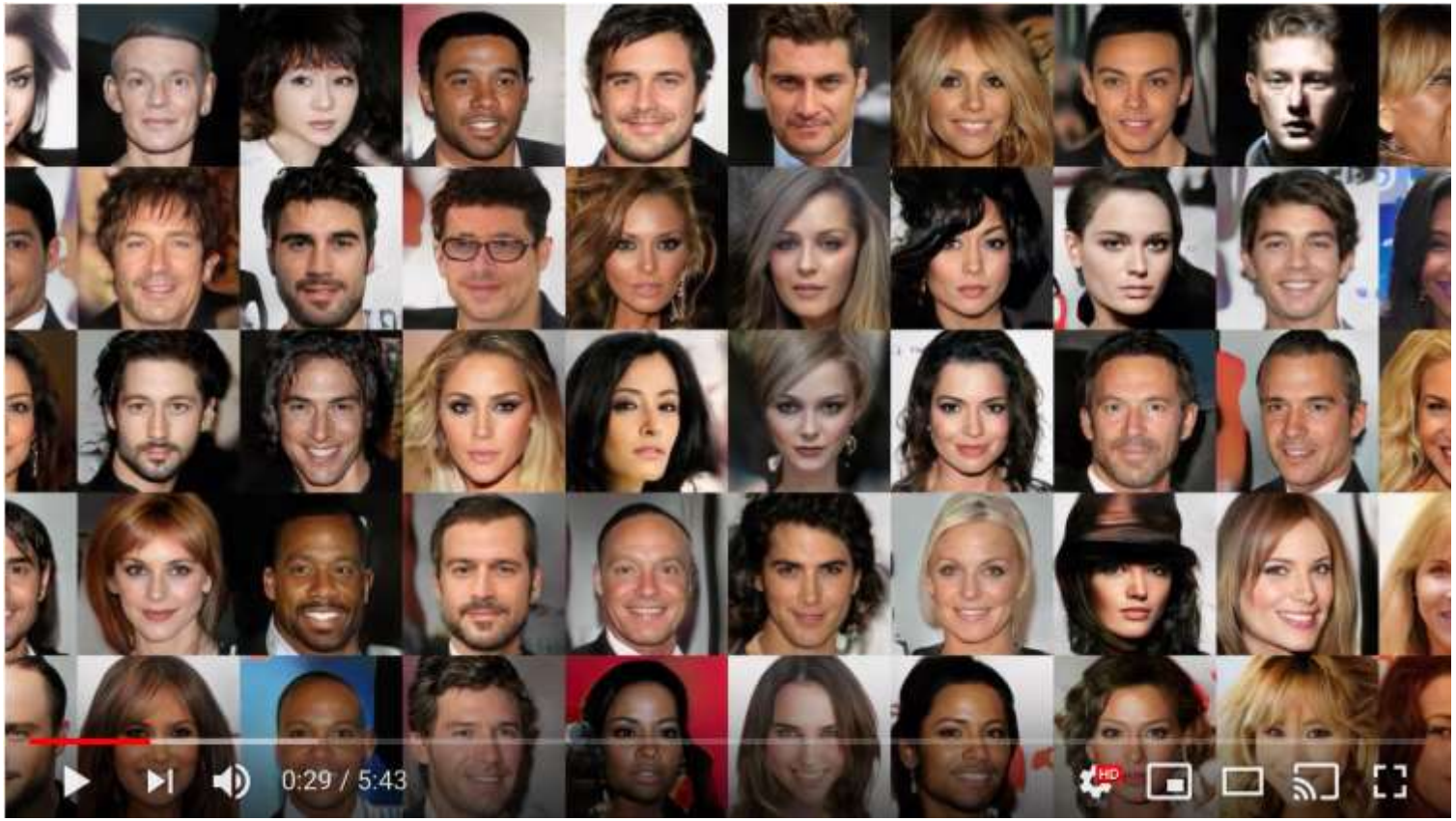
Previously unseen captions (*zero-shot* setting)



Captions seen in the training set

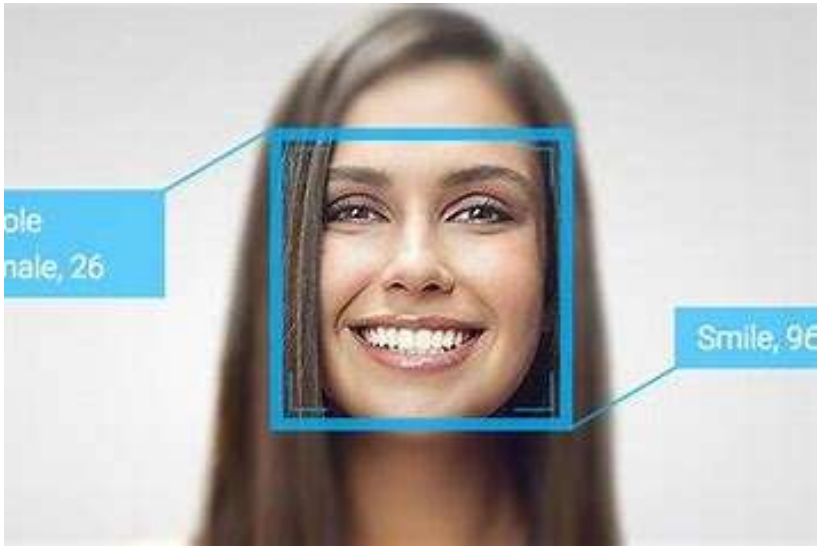
S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, H. Lee, Generative adversarial text to image synthesis, ICML 2016

Application: face generation

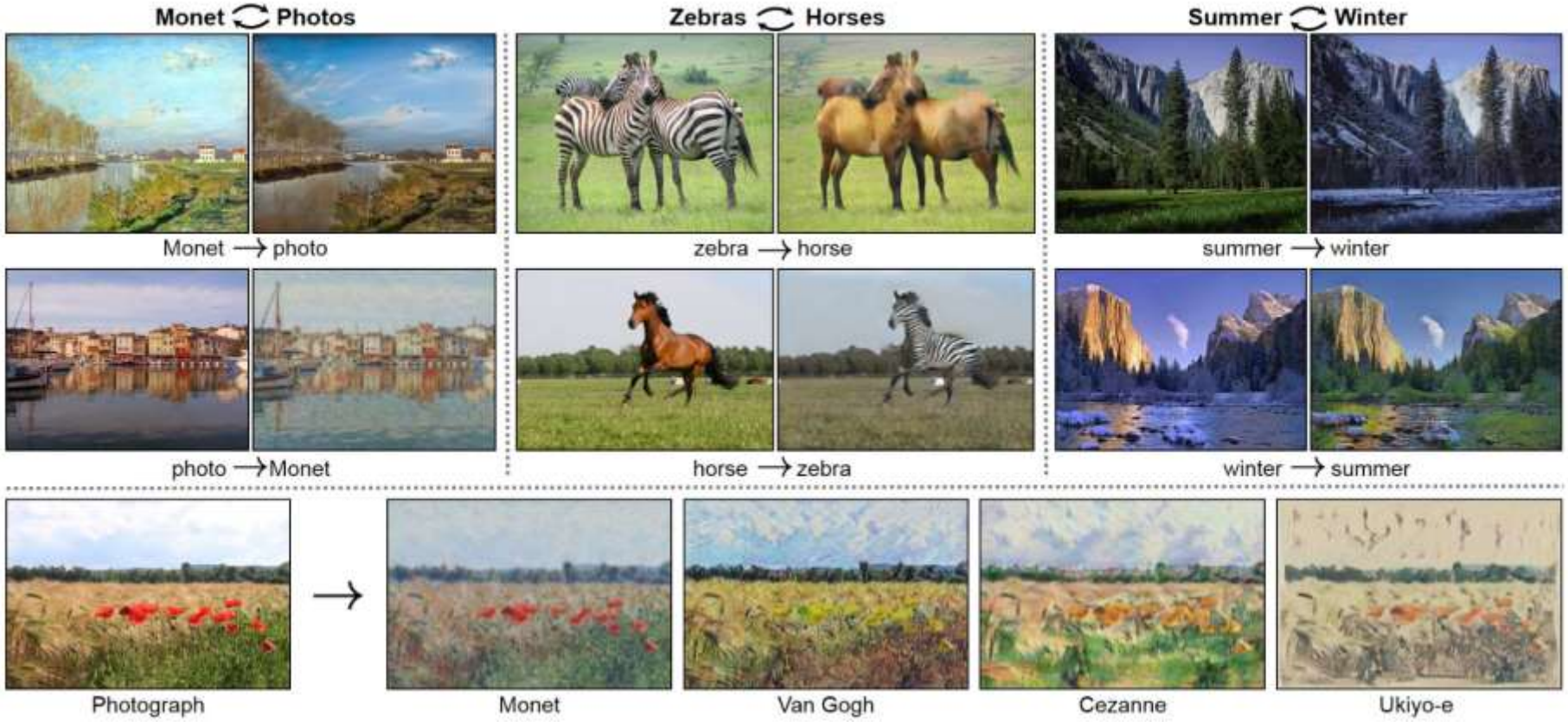


Source: https://www.youtube.com/watch?v=G06dEcZ-QTg


1024x1024
14 days 11 hours
9.63M reals shown

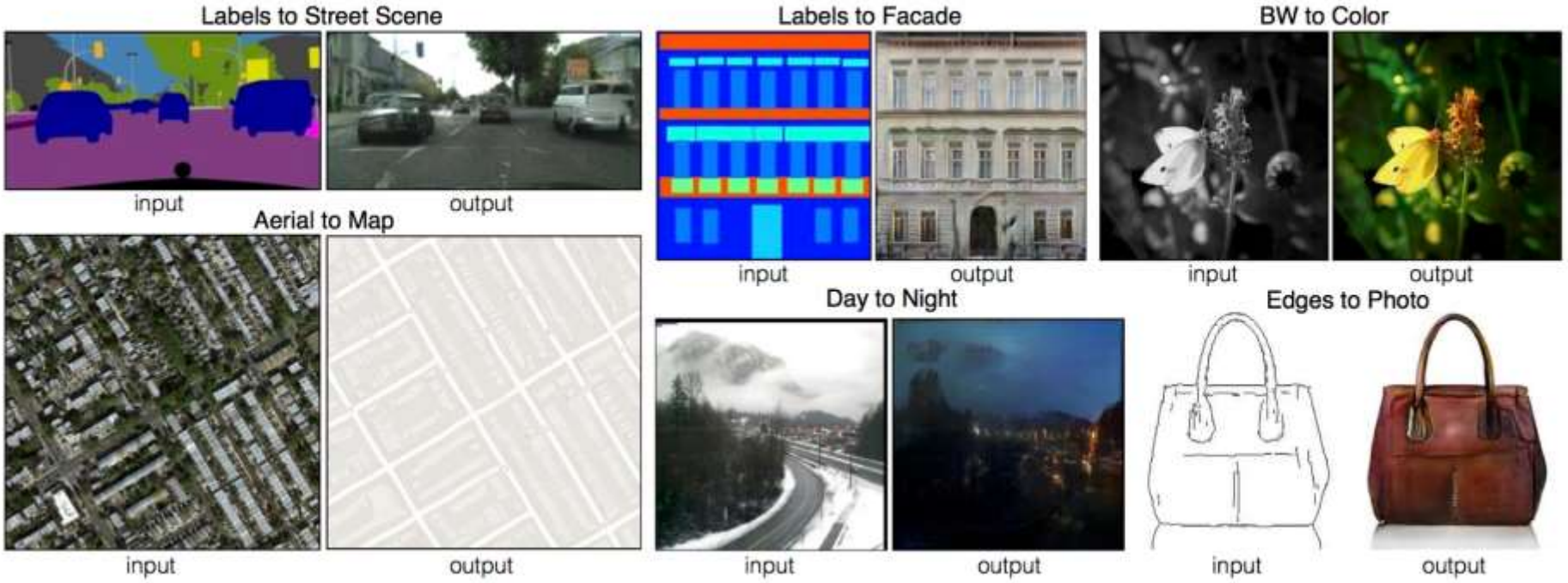

ple
nale, 26
Smile, 96

Application: style transfer



Source: https://github.com/junyanz/CycleGAN

Application: image translation



Source: https://github.com/phillipi/pix2pix

Conclusion

- GANs provide an attractive way to create a generative model without using an explicit encode (as in variational auto-encoders) but using supervised learning
- However the training is far from trivial and in general finding Nash equilibria in high-dimensional non-convex games is still an important open research problem